

Simulation d'une chaîne de transmission numérique

Rapport – Étapes 3 à 5

Canal gaussien, trajets multiples, récepteur à filtre adapté et codage de canal

Groupe : B4 – FIP2A
Auteurs : BLOMBOU Ethan
 BOUABOUD Anis-Melwan
 NANDA Laurent
 SISSOKO Mohamed Djoncounda
 ZIANI Mohamed Amine
Date : Septembre 2026

Table des matières

I	Étape 3 – Canal à bruit blanc additif gaussien	4
1	Objectifs de l'étape 3	4
2	Chaîne implémentée	4
3	Prévisions avant simulation	5
3.1	Paramètres pertinents	5
3.2	Résultats attendus	5
4	Modélisation du canal gaussien	5
4.1	Réglage de la puissance du bruit	5
4.2	Génération du bruit	5
4.3	Classe <code>TransmetteurAnalogiqueBruite</code>	6
4.4	Nouvelle option du simulateur	6
4.5	Mise en conformité de l'analyse des arguments	6
5	Validation du générateur de bruit	6
6	Résultats	7
6.1	Visualisation des signaux	7
6.2	TEB en fonction de E_b/N_0	7
7	Confrontation aux prévisions	9
7.1	Ce qui est conforme	9
7.2	Écart par rapport à la borne optimale	9
7.3	Classement des formes d'onde	9
8	Piste d'amélioration : récepteur moyenneur	9
9	Tests	10
9.1	Tests automatiques (<code>runTests</code>)	10
9.2	Tests unitaires JUnit	10
9.3	Limites identifiées	10
10	Conclusion	11
II	Étape 4 – Canal à trajets multiples et récepteur à filtre adapté	11
11	Objectifs de l'étape 4	11
12	Chaîne implémentée	11

13 Récapitulatif des modifications depuis l'étape 3	12
14 Prévisions avant simulation	12
14.1 Paramètres pertinents	12
14.2 Résultats attendus	13
15 Modélisation du canal à trajets multiples	13
15.1 Modèle discret	13
15.2 Option <code>-ti</code> de la commande unique	13
15.3 Implémentation par héritage	14
15.4 Défauts corrigés dans la première version	14
16 Récepteur à filtre adapté	15
16.1 Principe	15
16.2 Une règle de décision unique pour les trois formes d'onde	15
16.3 Cas du NRZT	15
16.4 Implémentation	16
16.5 TEB théorique	17
17 Résultats	17
17.1 Filtre adapté : conformité à la théorie	17
17.2 Gain par rapport à l'étape 3	18
17.3 Trajets multiples sans bruit (étape 4a)	19
17.4 Trajets multiples et bruit (étape 4b)	20
18 Confrontation aux prévisions	22
18.1 Ce qui est conforme	22
18.2 Ce qui diffère de l'intuition	22
19 Tests	22
19.1 Tests automatiques (<code>runTests</code>)	22
19.2 Tests unitaires JUnit	22
19.3 Limites identifiées	23
20 Conclusion	23
 III Étape 5 – Codage de canal	 23
21 Objectifs de l'étape 5	24
22 Chaîne implémentée	24
23 Récapitulatif des modifications depuis l'étape 4	25

24 Prévisions avant simulation	25
24.1 Paramètres pertinents	25
24.2 Résultats attendus	25
25 Le codage retenu	26
25.1 Mots de code et décodage	26
25.2 Lecture en termes d'automate	26
25.3 Intégration dans le simulateur	27
25.4 TEB théorique après décodage	27
26 Résultats	27
26.1 Amélioration du TEB à $-snr_{pb}$ égal	27
26.2 Comparaison à énergie égale par bit d'information	28
26.3 Codage et trajets multiples	29
26.4 Synchronisation émetteur/récepteur	30
27 Désynchronisation du récepteur (question 3 du TP5)	30
27.1 Modèle et réalisation	30
27.2 Effet sur les signaux	31
27.3 Effet sur le TEB sans bruit	31
27.4 Effet sur le TEB avec bruit	33
27.5 Le code permet de détecter une désynchronisation	34
27.6 Réponse à la question 3	34
28 Confrontation aux prévisions	35
28.1 Ce qui est conforme	35
28.2 Ce qui diffère de l'intuition	35
29 Tests	35
29.1 Tests automatiques (<code>runTests</code>)	35
29.2 Tests unitaires JUnit	36
30 Performances du simulateur	36
30.1 Constat	36
30.2 Modification réalisée : le signal rangé dans des tableaux de <code>float</code>	37
30.3 Autres pistes	38
31 Conclusion générale	38

Organisation du rapport

Ce rapport est cumulatif. La **partie I** reprend sans modification le rapport de l'étape 3 (canal à bruit blanc additif gaussien). La **partie II** présente l'étape 4 : le canal à trajets multiples et le récepteur à filtre adapté, qui lève la principale limite relevée à l'étape 3. La **partie III** présente l'étape 5 : le codage de canal, puis un bilan des performances du simulateur et une conclusion générale. Chaque partie suit le même plan : objectifs, prévisions, réalisation, résultats, confrontation aux prévisions, tests.

Partie I

Étape 3 – Canal à bruit blanc additif gaussien

1 Objectifs de l'étape 3

L'étape 3 remplace le canal analogique parfait de l'étape 2 par un canal à **bruit blanc additif gaussien** (BBAG, ou AWGN). Le signal émis n'arrive plus intact au récepteur : des erreurs de décision apparaissent et le taux d'erreur binaire (TEB) devient non nul. L'objectif est donc de :

- modéliser le canal $r(n) = s(n) + b(n)$ avec $b(n) \sim \mathcal{N}(0, \sigma_b^2)$;
- régler σ_b^2 à partir du rapport signal à bruit par bit E_b/N_0 fourni en paramètre ;
- vérifier que le bruit généré est bien gaussien ;
- tracer le TEB en fonction de E_b/N_0 et le confronter aux prévisions théoriques.

2 Chaîne implémentée

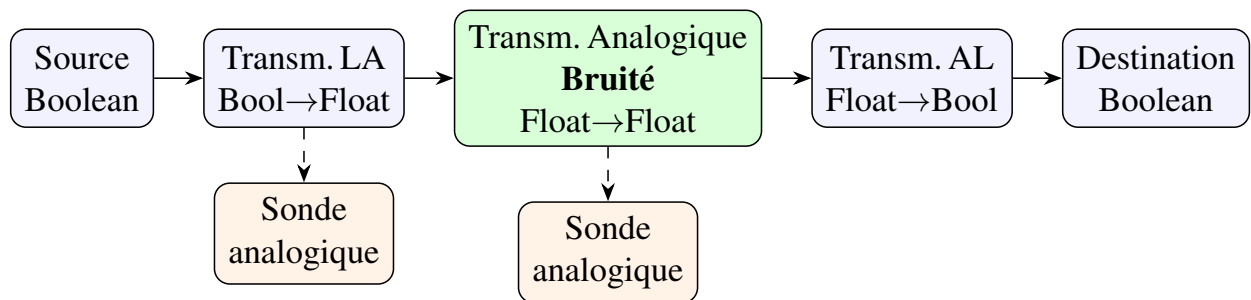


Figure 1: Chaîne de l'étape 3 – seul le canal (en vert) change par rapport à l'étape 2.

L'architecture en composants connectables mise en place aux étapes 1 et 2 a permis d'introduire le bruit sans modifier ni l'émetteur (`TransmetteurLogiqueAnalogique`) ni le récepteur (`TransmetteurAnalogiqueLogique`) : le simulateur instancie simplement un `TransmetteurAnalogiqueBruite` à la place du `TransmetteurAnalogiqueParfait` lorsque l'option `-snrpb` est présente.

3 Prévisions avant simulation

3.1 Paramètres pertinents

Paramètre	Option	Rôle
Rapport E_b/N_0 (dB)	-snrpb	fixe la puissance du bruit
Forme d'onde	-form	NRZ, NRZT ou RZ
Nombre d'échantillons par bit N	-nbEch	intervient dans le calcul de σ_b^2
Amplitudes $A_{\min}, A_{\max}$	-ampl	fixent la puissance P_s et le seuil
Longueur du message	-mess	précision statistique du TEB
Semence	-seed	reproductibilité (message et bruit)

3.2 Résultats attendus

- Le TEB doit décroître de façon monotone quand E_b/N_0 augmente, et tendre vers 0,5 quand $E_b/N_0 \rightarrow -\infty$.
- Le TEB doit être indépendant de l'amplitude absolue du signal : multiplier $A_{\min}$ et $A_{\max}$ par un même facteur multiplie P_s , donc σ_b , dans le même rapport.
- Pour un récepteur optimal (filtre adapté), la théorie donne $\text{TEB} = Q(\sqrt{2E_b/N_0})$ en NRZ antipodal et $Q(\sqrt{E_b/N_0})$ pour un signal unipolaire (0, A), soit une pénalité de 3 dB pour l'unipolaire.
- Notre récepteur de l'étape 2 ne prend qu'un seul échantillon par bit. On s'attend donc à des performances inférieures à la borne optimale ; la section 7 quantifie cet écart.

4 Modélisation du canal gaussien

4.1 Réglage de la puissance du bruit

D'après la séance 3, en notant P_s la puissance moyenne du signal, T la durée d'un bit, F_e la fréquence d'échantillonnage et $N = TF_e$ le nombre d'échantillons par bit :

$$E_b = P_s T, \quad N_0 = \frac{2\sigma_b^2}{F_e} \implies \frac{E_b}{N_0} = \frac{P_s N}{2\sigma_b^2}.$$

On inverse cette relation pour obtenir la variance du bruit à appliquer :

$$\sigma_b^2 = \frac{P_s N}{2 \cdot 10^{(E_b/N_0)_{\text{dB}}/10}}$$

La puissance P_s est estimée directement sur le signal reçu par le canal, $P_s = \frac{1}{K} \sum_{n=1}^K s^2(n)$, ce qui la rend automatiquement correcte quelle que soit la forme d'onde et les amplitudes choisies.

4.2 Génération du bruit

Chaque échantillon de bruit est obtenu à partir de deux tirages uniformes indépendants $a_1, a_2 \sim \mathcal{U}[0, 1[$ (transformation de Box–Muller donnée en cours) :

$$b(n) = \sigma_b \sqrt{-2 \ln(1 - a_1(n))} \cos(2\pi a_2(n)).$$

L'utilisation de $1 - a_1$ plutôt que a_1 évite le calcul de $\ln 0$, puisque `nextDouble()` peut renvoyer exactement 0.

4.3 Classe `TransmetteurAnalogiqueBruit`

- Hérite de `Transmetteur<Float, Float>`.
- Constructeur `(int nbEch, float ebN0Db, Integer seed)` : le générateur `java.util.Random` est initialisé avec la semence si l'option `-seed` est fournie, ce qui permet de rejouer une simulation à l'identique.
- `emettre()` : calcule P_s , en déduit σ_b , ajoute un échantillon de bruit à chaque échantillon du signal puis transmet le résultat aux destinations connectées. Un message vide est retransmis tel quel.

4.4 Nouvelle option du simulateur

Option	Description
<code>-snrpb s</code>	canal bruité, $E_b/N_0 = s$ dB (flottant, peut être négatif) ; active le mode analogique

Sans cette option, le canal reste parfait et le comportement des étapes 1 et 2 est inchangé. Le nom de l'option est celui imposé par la commande unique.

4.5 Mise en conformité de l'analyse des arguments

À l'occasion de cette étape, l'analyse des arguments a été alignée sur la commande unique : l'option `-ampl min max` lève désormais une `ArgumentsException` si $\min \geq \max$ (auparavant, `-ampl 1 0` était accepté et produisait un TEB de 1). Nous conservons en revanche un minimum de 10 échantillons par bit pour `-nbEch` : en dessous, le tiers central du RZ et la rampe du NRZT tiennent sur 0 ou 1 échantillon et la forme d'onde n'est plus correctement représentée. Ce choix est documenté dans le README.

5 Validation du générateur de bruit

Pour vérifier le générateur (remarque de la séance 3), on a envoyé dans le canal un signal constant égal à 1 ($P_s = 1$) sur 500 000 échantillons avec $N = 30$ et $E_b/N_0 = 10$ dB, puis soustrait le signal pour isoler le bruit.

	Attendu	Mesuré
Moyenne	0	$5,6 \times 10^{-4}$
Écart-type σ_b	$\sqrt{30/(2 \cdot 10)} = 1,2247$	1,2247

L'histogramme se superpose à la densité $\mathcal{N}(0, 1)$ et l'écart-type mesuré coïncide avec la valeur calculée à 10^{-4} près : le générateur et le réglage de σ_b sont validés.

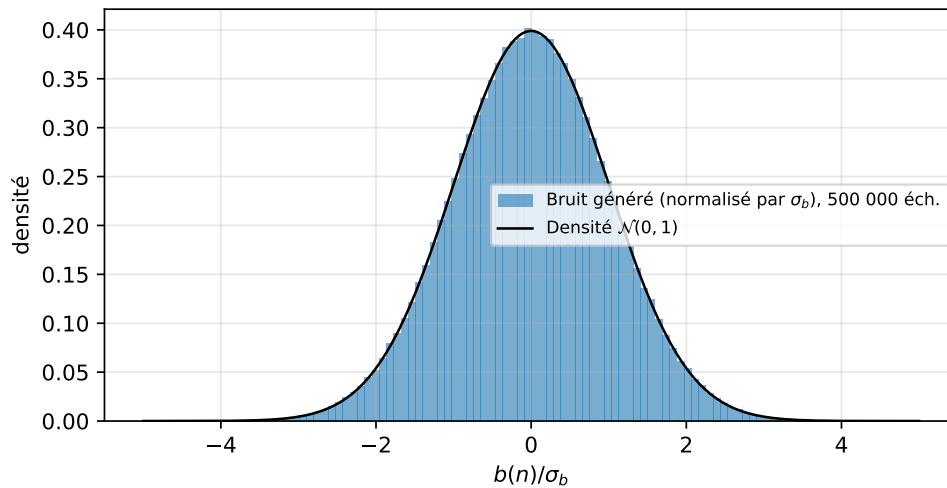


Figure 2: Histogramme du bruit généré comparé à la densité gaussienne centrée réduite.

6 Résultats

6.1 Visualisation des signaux

Avec $N = 30$, même à $E_b/N_0 = 25$ dB le bruit est bien visible sur chaque échantillon. À 15 dB, sa dispersion est du même ordre que l'amplitude du signal : un échantillon isolé peut franchir le seuil et provoquer une erreur de décision.

6.2 TEB en fonction de E_b/N_0

Les courbes ont été obtenues avec 200 000 bits par point ($N = 30$), ce qui permet de mesurer des TEB jusqu'à environ 10^{-5} . Les points à TEB nul ne sont pas représentés.

Quelques exécutions du simulateur en ligne de commande (100 000 bits, $N = 30$) :

Commande	TEB
<code>-mess 100000 -form NRZ -ampl -1 1 -snrpb 8 -seed 1</code>	0,259
<code>-mess 100000 -form NRZ -ampl -1 1 -snrpb 20 -seed 1</code>	$5,1 \times 10^{-3}$
<code>-mess 100000 -form NRZT -ampl -1 1 -snrpb 20 -seed 1</code>	$3,3 \times 10^{-3}$
<code>-mess 100000 -form RZ -snrpb 20 -seed 1</code>	$9,8 \times 10^{-4}$
<code>-mess 100000 -form NRZ -ampl -1 1 -snrpb 30 -seed 1</code>	0

Deux exécutions avec la même semence donnent exactement le même TEB, ce qui confirme la reproductibilité.

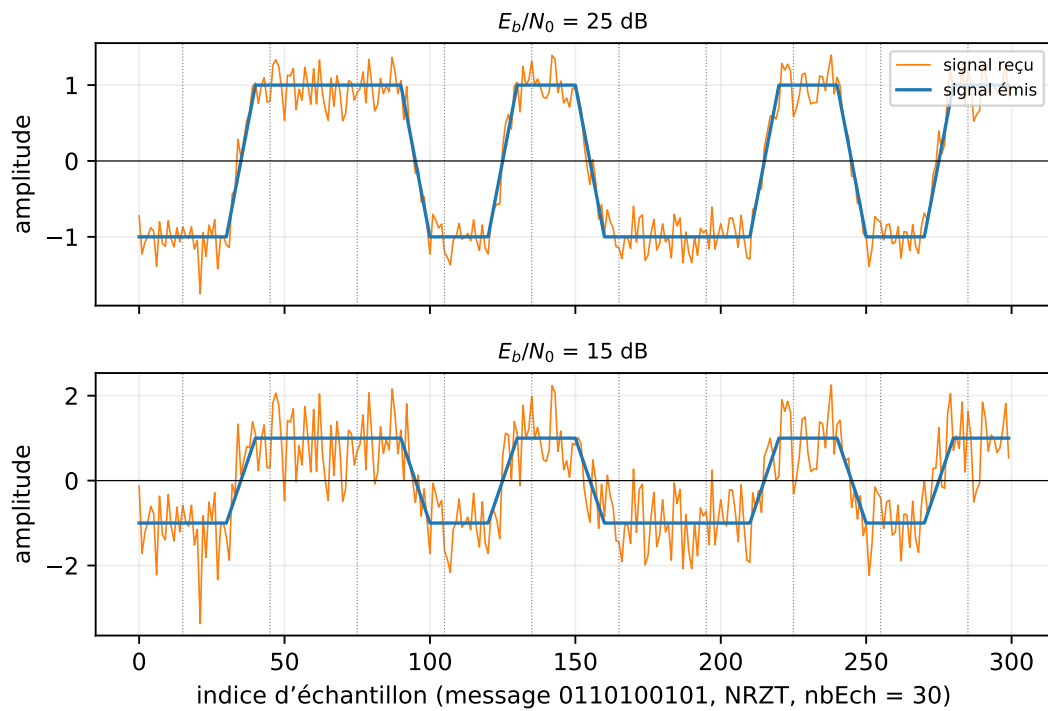


Figure 3: Signal NRZT émis et reçu pour deux valeurs de E_b/N_0 . Les pointillés marquent l'échantillon central sur lequel le récepteur prend sa décision.

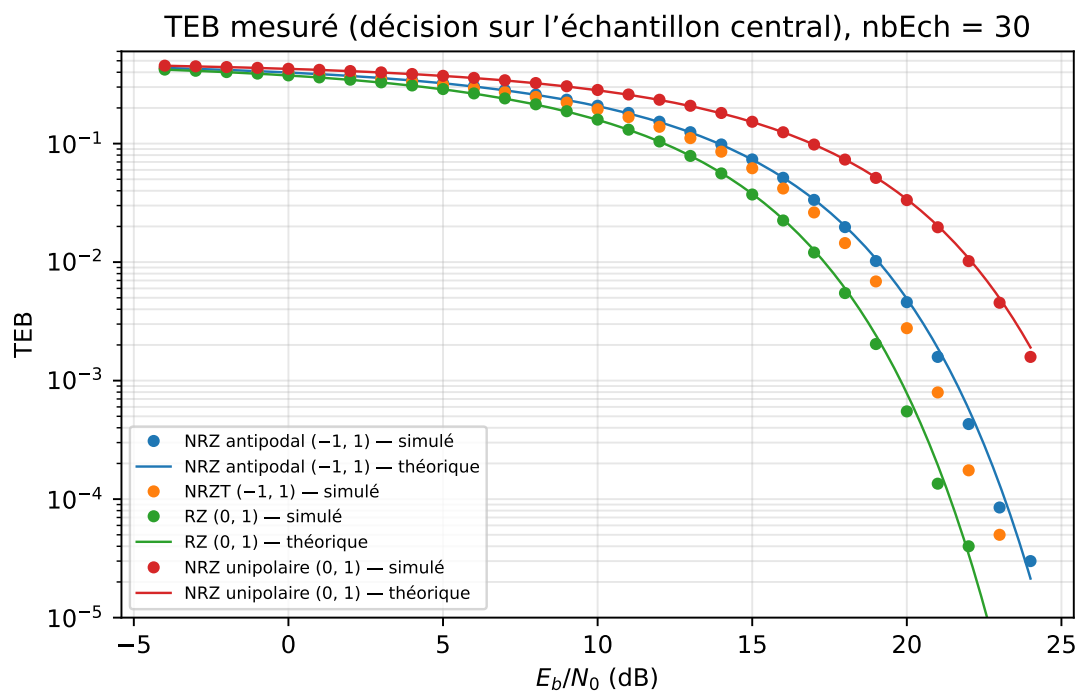


Figure 4: TEB simulé par notre chaîne (points) et TEB théorique pour une décision sur un échantillon (traits).

7 Confrontation aux prévisions

7.1 Ce qui est conforme

Le TEB décroît bien de façon monotone avec E_b/N_0 et tend vers 0,5 aux faibles rapports. Il ne dépend pas de l'amplitude absolue, conformément à la normalisation par P_s .

7.2 Écart par rapport à la borne optimale

En revanche, les TEB mesurés sont très supérieurs à la prévision $Q(\sqrt{2E_b/N_0})$: à 8 dB, on attendait environ 2×10^{-4} en NRZ antipodal et l'on mesure 0,26. L'écart vient du récepteur, pas du canal.

Le récepteur décide à partir d'un **seul** échantillon, dont le rapport signal à bruit vaut, pour un NRZ antipodal ($\pm A$, $P_s = A^2$) :

$$\frac{A^2}{\sigma_b^2} = \frac{A^2 \cdot 2E_b/N_0}{A^2 N} = \frac{2E_b}{N N_0} \implies \text{TEB} = Q\left(\sqrt{\frac{2E_b}{N N_0}}\right).$$

L'énergie du bit est répartie sur N échantillons, mais le récepteur n'en exploite qu'un : il perd un facteur N , soit $10 \log_{10} 30 \approx 14,8$ dB. Le même raisonnement donne :

Forme	P_s	Distance au seuil	TEB (1 échantillon)
NRZ antipodal ($-A, A$)	A^2	A	$Q(\sqrt{2E_b/N_0/N})$
NRZ unipolaire ($0, A$)	$A^2/2$	$A/2$	$Q(\sqrt{E_b/N_0/N})$
RZ ($0, A$)	$A^2/6$	$A/2$	$Q(\sqrt{3E_b/N_0/N})$

Ces expressions (traits de la figure 4) se superposent aux points simulés, ce qui valide à la fois le canal et notre compréhension du récepteur.

7.3 Classement des formes d'onde

- **NRZ unipolaire vs antipodal** : 3 dB d'écart, comme prévu par la théorie.
- **NRZT** : légèrement meilleur que le NRZ (environ 0,5 dB). Les rampes réduisent la puissance moyenne ($P_s \approx 0,89A^2$), donc σ_b , alors que l'échantillon central reste sur le plateau à $\pm A$.
- **RZ** : meilleur que le NRZ antipodal avec ce récepteur (environ 1,8 dB), ce qui contredit l'intuition. C'est un artefact de la décision ponctuelle : le signal RZ est nul les deux tiers du temps, sa puissance moyenne est faible, et le bruit calibré sur E_b/N_0 l'est aussi, alors que l'échantillon central tombe précisément sur l'impulsion. Avec un récepteur optimal, le RZ unipolaire retrouve sa place, 3 dB derrière le NRZ antipodal.

8 Piste d'amélioration : récepteur moyennneur

Pour vérifier l'analyse précédente, nous avons testé hors du simulateur, à l'aide d'un banc de mesure réutilisant nos classes `SourceAleatoire`, `TransmetteurLogiqueAnalogique` et `TransmetteurAnalogiqueBruite`, une décision sur la **moyenne des échantillons** du temps bit (sur tout le bit en NRZ, sur le tiers central en RZ). Moyenner M échantillons divise la variance du bruit par M sans changer le niveau utile ; c'est l'équivalent discret du filtre adapté pour une impulsion rectangulaire. Le récepteur moyennneur atteint la borne $Q(\sqrt{2E_b/N_0})$ (TEB mesuré de $2,05 \times 10^{-4}$ à 8 dB pour $1,9 \times 10^{-4}$ attendu), soit un gain de 14,8 dB. En RZ, moyenner sur le tiers central donne $Q(\sqrt{E_b/N_0})$ ($5,7 \times 10^{-3}$ mesuré à 8 dB pour 6×10^{-3} attendu). Ce récepteur n'est pas encore intégré dans `TransmetteurAnalogiqueLogique` ; il le sera dans une prochaine version, avec une option permettant de conserver la décision ponctuelle pour comparaison.

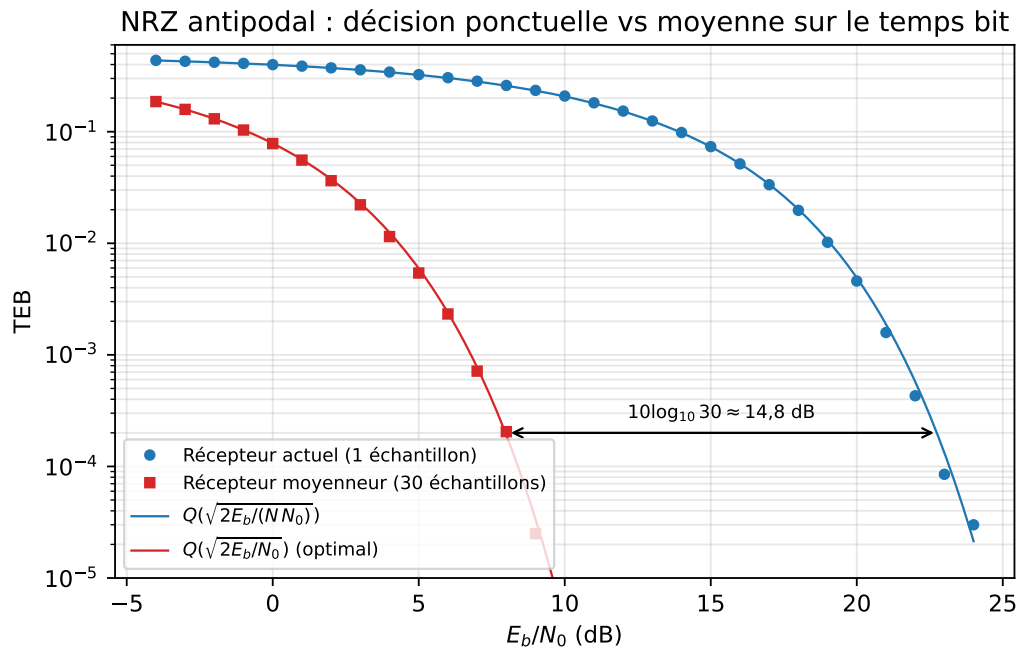


Figure 5: NRZ antipodal : récepteur actuel et récepteur moyenneur, 200 000 bits par point.

9 Tests

9.1 Tests automatiques (runTests)

Neuf tests ont été ajoutés pour l'étape 3 : cinq exécutions nominales (E_b/N_0 élevé et faible, chaîne NRZT complète, `-snrpb` seul, performance sur 10 000 bits limitée à 10 s) et quatre tests d'erreur sur les nouveaux contrôles d'arguments.

```
Test 10 : Bruit (Eb/N0 élevé, peu d'erreurs) ... OK
Test 11 : Bruit (Eb/N0 faible, beaucoup d'erreurs) ... OK
Test 12 : Bruit complet (NRZT + ampl + snrpb) ... OK
Test 13 : Bruit seul (-snrpb active le mode analogique) ... OK
Test 14 : Performance (message aléatoire de 10000 bits) ... OK
Test 19 : Erreur (ampl min > max) ... OK
Test 20 : Erreur (ampl min = max) ... OK
Test 21 : Erreur (snrpb non flottant) ... OK
Test 22 : Erreur (snrpb sans valeur) ... OK
```

9.2 Tests unitaires JUnit

Six tests ont été ajoutés à `SimulateurTest` : refus de `-ampl` avec `min > max` et `min = max`, TEB non nul à 0 dB, TEB nul à 40 dB, refus d'une valeur non flottante pour `-snrpb` et refus de l'ancienne option `-ebn0`. L'ensemble de la suite passe : 75 tests sur 75.

9.3 Limites identifiées

- Ces tests vérifient que le simulateur s'exécute et affiche un TEB, pas la valeur du TEB. Un test de non-régression comparant le TEB mesuré à la valeur théorique, avec une tolérance, serait plus robuste.

- Avec `-seed`, la source et le canal sont initialisés avec la **même** semence. Les deux générateurs produisent alors des suites liées. Utiliser deux semences dérivées (par exemple `seed` et `seed + 1`) garantirait l'indépendance du message et du bruit.

10 Conclusion

Le canal à bruit blanc additif gaussien est fonctionnel et validé : le bruit généré est gaussien, centré, et son écart-type correspond exactement à la valeur déduite de E_b/N_0 . Les TEB mesurés concordent avec la théorie pour le récepteur actuel, qui décide sur un seul échantillon par bit.

La confrontation aux prévisions met en évidence la principale limite de la chaîne : ce récepteur perd $10\log_{10}N \approx 14,8$ dB par rapport à l'optimum. Un récepteur moyennneur, testé sur banc, atteint la borne théorique. Son intégration est la priorité avant l'étape 5b, qui cherchera à réduire le TEB par codage de canal : il serait incohérent de coder le canal avant d'avoir récupéré ces 15 dB à la réception.

Partie II

Étape 4 – Canal à trajets multiples et récepteur à filtre adapté

11 Objectifs de l'étape 4

L'énoncé de l'atelier découpe l'étape 4 en deux parties :

- **4a** – transmission non idéale avec un bruit « réel » : nous avons retenu les **trajets multiples**, décrits par la fiche de la séance 4. Le signal reçu est la somme du trajet direct et de copies retardées et atténuées :

$$r(n) = s(n) + \sum_{k=1}^K ar_k s(n - dt_k), \quad K \leq 5;$$

- **4b** – les mêmes trajets multiples **plus** le bruit blanc gaussien de l'étape 3 : $r(n) = s(n) + \sum_k ar_k s(n - dt_k) + b(n)$.

Le rapport de l'étape 3 concluait par ailleurs que notre récepteur, qui décidait sur un seul échantillon par bit, perdait $10\log_{10}N \approx 14,8$ dB par rapport à l'optimum, et que l'intégration d'un récepteur optimal était la priorité avant le codage de canal. Cette étape réalise donc aussi le **récepteur à filtre adapté**. Les objectifs sont :

- modéliser le canal à trajets multiples, avec ou sans bruit, et l'intégrer au simulateur via l'option `-ti` de la commande unique ;
- remplacer la décision sur l'échantillon central par un filtre adapté valable pour les trois formes d'onde ;
- mesurer le TEB dans chaque configuration et le confronter à une théorie exacte de la chaîne.

12 Chaîne implémentée

Le simulateur choisit le canal selon les options présentes :

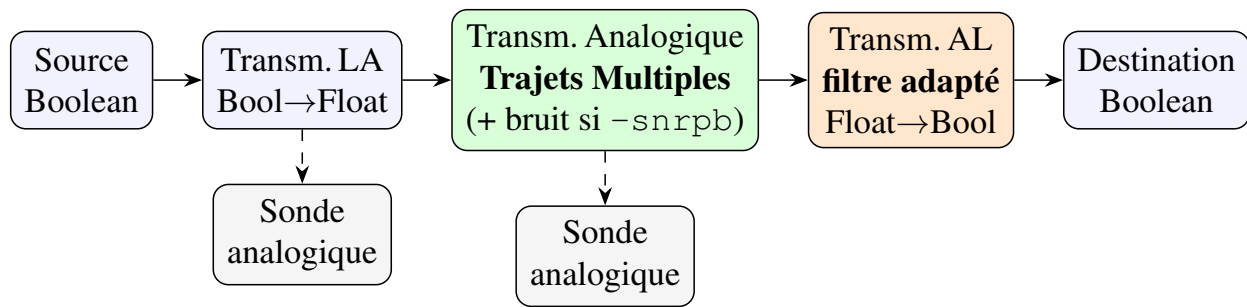


Figure 6: Chaîne de l'étape 4. En vert, le nouveau canal ; en orange, le récepteur modifié.

Options	Canal instancié	Perturbation
aucune	TransmetteurAnalogiqueParfait	aucune
-snrpb s	TransmetteurAnalogiqueBruite	bruit gaussien (étape 3)
-ti dt ar ...	TransmetteurAnalogiqueTrajetsMultiples	échos seuls (4a)
-ti dt ar ... -snrpb s	TransmetteurAnalogiqueTrajetsMultiples	échos puis bruit (4b)

Le récepteur à filtre adapté est utilisé dans tous les cas analogiques.

13 Récapitulatif des modifications depuis l'étape 3

Élément	Modification
TransmetteurAnalogiqueTrajetsMultiples	Nouvelle classe. Canal à trajets multiples, bruité ou non. Hérite de TransmetteurAnalogiqueBruite (section 15.3).
TransmetteurAnalogiqueBruite	Le calcul du signal auquel on ajoute le bruit est isolé dans une méthode protégée <code>signalAvantBruit()</code> , que la sous-classe redéfinit. Comportement inchangé pour le canal gaussien seul.
TransmetteurAnalogiqueLogique	Récepteur à filtre adapté : corrélation de tout le temps bit avec $g = s_1 - s_0$ au lieu d'un seuil sur l'échantillon central (section 16). Le constructeur reçoit désormais la forme d'onde.
TransmetteurLogiqueAnalogiqueSimulateur	La génération d'un bit est extraite dans <code>formeBit(...)</code> , partagée avec le récepteur. Signal émis inchangé.
runTests	Nouvelle option <code>-ti</code> ; sélection du canal ; E_b/N_0 infini sans <code>-snrpb</code> ; forme d'onde transmise au récepteur.
Tests JUnit	Section « Étape 4 » : 3 exécutions nominales et 3 tests d'erreur sur <code>-ti</code> ; renumérotation des commentaires (28 tests).
genDeliverable, README	22 tests ajoutés, 1 supprimé (option <code>-ebn0</code> hors spécification) : 96 tests, tous réussis.
rapport/etape4/	Archive <code>etape4</code> ; documentation de <code>-ti</code> et du filtre adapté, exemples remesurés.
	Script <code>courbesTEB.py</code> (simulations et théorie exacte) et outil <code>ExportSignal.java</code> qui produisent toutes les figures de ce rapport.

14 Prévisions avant simulation

14.1 Paramètres pertinents

Paramètre	Option	Rôle
Retard dt_k d'un trajet indirect (échantillons)	-ti	position de l'écho par rapport au bit
Amplitude relative ar_k	-ti	force de l'écho
Nombre de trajets (1 à 5)	-ti	nombre de couples $dt\ ar$
Rapport E_b/N_0 (dB)	-snrpb	bruit ajouté après les échos (4b)
Forme d'onde, N , amplitudes	-form, -nbEch, -ampl	forme du filtre adapté et du seuil

14.2 Résultats attendus

- **Filtre adapté.** Le TEB doit rejoindre la borne optimale : $Q(\sqrt{2E_b/N_0})$ en NRZ antipodal et $Q(\sqrt{E_b/N_0})$ en unipolaire, soit un gain de $10\log_{10} N \approx 14,8$ dB sur l'étape 3.
- **Trajets multiples sans bruit (4a).** L'écho d'un bit « déborde » sur les bits suivants : c'est de l'interférence entre symboles (IES). Tant qu'elle ne fait pas franchir le seuil, le TEB reste nul ; au-delà, le TEB passe brusquement à une fraction fixe (les suites de bits défavorables). Un écho retardé d'un bit entier ($dt = N$) doit être le plus gênant.
- **Trajets multiples et bruit (4b).** Pour un même E_b/N_0 , le TEB doit être plus élevé qu'en l'absence d'écho, et d'autant plus que ar est grand.

15 Modélisation du canal à trajets multiples

15.1 Modèle discret

Chaque trajet indirect k est une copie du signal émis, décalée de dt_k échantillons et multipliée par ar_k :

$$r(n) = s(n) + \sum_{k=1}^K ar_k s(n - dt_k) \quad (+b(n) \text{ en 4b}), \quad s(m) = 0 \text{ pour } m < 0.$$

Avant le début d'un écho ($n < dt_k$), rien n'est encore réfléchi. La longueur du signal est conservée : la queue de l'écho du dernier bit est tronquée, ce qui est sans effet sur la décision.

15.2 Option -ti de la commande unique

La commande unique impose la syntaxe `-ti dt ar [dt ar ...]` : le retard (entier) vient avant l'amplitude (flottant), jusqu'à 5 couples, sans indiquer leur nombre. L'analyse lit des couples tant que l'argument suivant est un entier :

```

} else if (args[i].matches("-ti")) {
    // Syntaxe : -ti dt1 ar1 [dt2 ar2 ...] (5 couples au maximum)
    simulationAnalogique = true;
    canalTrajetsMultiples = true;
    while (i + 1 < args.length && args[i + 1].matches("-?[0-9]+")) {
        i++; int dt = Integer.parseInt(args[i]); // dt >= 0, sinon erreur
        i++; amplitudes.add(Float.parseFloat(args[i])); // ar manquant ou non
            flottant : erreur
        retards.add(dt);
    }
    if (retards.isEmpty() || retards.size() > NB_TRAJETS_MAX) throw new
        ArgumentsException(...);

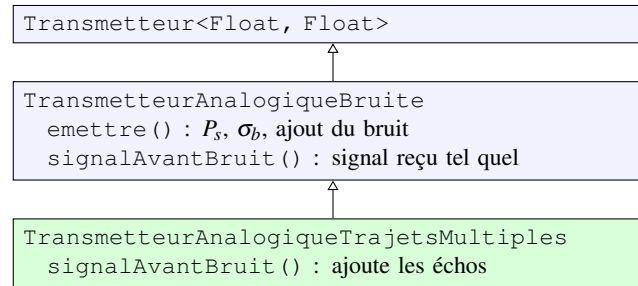
```

Les arguments invalides lèvent une `ArgumentsException` : -ti sans couple, plus de 5 couples, ar manquant ou non numérique, dt négatif ou hors des entiers. Un dt négatif est refusé car il désignerait

un écho arrivant *avant* le trajet direct ; il provoquait de plus un accès hors du tableau. Comme `-form` ou `-snrpb`, `-ti` active la simulation analogique.

15.3 Implémentation par héritage

Un canal à trajets multiples bruité est un canal gaussien dans lequel on insère une étape avant l'ajout du bruit. Plutôt que de dupliquer le calcul du bruit, la nouvelle classe hérite de `TransmetteurAnalogiqueBruite`, selon le patron « méthode modèle » :



```

// TransmetteurAnalogiqueBruite.emettre() (extrait simplifié)
float[] signal = signalAvantBruit(); // point d'extension
double ps = sommeCarres(signal) / n; // puissance mesurée après les échos
double sigmaB2 = (ps * nbEch) / (2.0 * ebN0Lineaire);
for (int i = 0; i < n; i++)
    informationEmise.add((float) (signal[i] + genererBruitGaussien(sigmaB)));

// TransmetteurAnalogiqueTrajetsMultiples
@Override
protected float[] signalAvantBruit() {
    for (int i = 0; i < n; i++) {
        float valeur = informationRecue.iemeElement(i);
        for (int k = 0; k < retards.length; k++)
            if (i - retards[k] >= 0)
                valeur += amplitudes[k] * informationRecue.iemeElement(i -
                    retards[k]);
        avecTrajets[i] = valeur;
    }
    return avecTrajets;
}

```

Conséquences :

- la formule du bruit n'existe qu'à un seul endroit, ce qui garantit qu'un même `-snrpb` donne le même bruit dans les deux canaux ;
- conformément à la fiche de la séance 4, $P_s = \frac{1}{N} \sum s_{tj}^2$ est mesurée sur le signal *avec échos* ;
- sans `-snrpb`, le simulateur transmet $E_b/N_0 = +\infty$: on obtient alors $\sigma_b = 0$ et aucun bruit n'est ajouté (étape 4a), sans aucun cas particulier dans le code.

15.4 Défauts corrigés dans la première version

La première version du canal (commit « Ajout du signal bruité ») a été relue et confrontée à la commande unique. Cinq défauts ont été corrigés :

Défaut	Effet mesuré	Correction
Option <code>-trajets n ar dt ...</code>	refus des commandes de l'enseignant (<code>-ti</code>)	syntaxe <code>-ti dt ar</code>
Condition <code>canalBruite</code> testée deux fois	<code>-trajets</code> sans effet	branchement corrigé
E_b/N_0 par défaut à 0 dB	écho $dt = N$, $ar = 0,5$, sans <code>-snrpb</code> (NRZ ± 1 , $N = 10$) : TEB 0,150 au lieu de 0	$E_b/N_0 = +\infty$
Facteur N oublié dans σ_b^2 (copie de la formule)	bruit 30 fois trop faible : écho nul, 0 dB (NRZ 0/1), TEB 0,158 au lieu de 0,428	héritage
Retard négatif accepté	accès hors du tableau	refusé

Le troisième défaut rendait un test JUnit trompeur : « TEB non nul avec `-trajets` sans `-snrpb` » ne réussissait que grâce au bruit caché. Il a été remplacé par deux tests, l'un avec des échos réellement destructeurs, l'autre vérifiant qu'un écho faible sans bruit donne un TEB nul.

16 Récepteur à filtre adapté

16.1 Principe

Pour une impulsion $p(t)$ de durée T reçue dans un bruit blanc gaussien, le filtre linéaire qui maximise le rapport signal à bruit à l'instant de décision est le **filtre adapté**, de réponse impulsionnelle $h(t) = p(T - t)$: l'impulsion retournée dans le temps et décalée pour être causale [1]. Échantillonnée en $t = T$, sa sortie est une **corrélation** :

$$y(T) = \int r(\tau) h(T - \tau) d\tau = \int_0^T r(\tau) p(\tau) d\tau.$$

Le signal utile s'additionne de façon cohérente sur tout le bit, alors que le bruit, indépendant d'un échantillon à l'autre, se moyenne. C'est précisément ce que ne faisait pas le récepteur de l'étape 3, qui n'utilisait qu'un échantillon sur N .

16.2 Une règle de décision unique pour les trois formes d'onde

Notons s_1 et s_0 les N échantillons d'un bit 1 et d'un bit 0, et r les échantillons reçus pendant un bit. Avec un bruit gaussien, la décision optimale choisit le symbole le plus proche :

$$\|r - s_1\|^2 < \|r - s_0\|^2 \iff \underbrace{\sum_{k=0}^{N-1} r[k] g[k]}_y > \underbrace{\frac{\|s_1\|^2 - \|s_0\|^2}{2}}_{\text{seuil}}, \quad g = s_1 - s_0.$$

C'est un filtre adapté à la *différence* g des deux symboles. La forme d'onde n'apparaît que dans g : la même formule convient à NRZ, NRZT et RZ, et le seuil s'en déduit automatiquement (figure 7) :

- **NRZ** : g est constant, y est la somme des échantillons du bit ; ramené à une moyenne, le seuil vaut $(A_{\min} + A_{\max})/2$ comme à l'étape 3 ;
- **RZ** : g est nul hors du tiers central, qui ne contient que du bruit ; le filtre l'ignore ;
- **NRZT** : les échantillons des rampes, moins fiables, reçoivent un poids plus faible.

16.3 Cas du NRZT

Dans notre émetteur, la rampe d'un bit part du niveau du bit précédent : un bit 1 précédé d'un 1 est plat, précédé d'un 0 il commence par une rampe montante. Le symbole n'est donc pas unique. Nous prenons

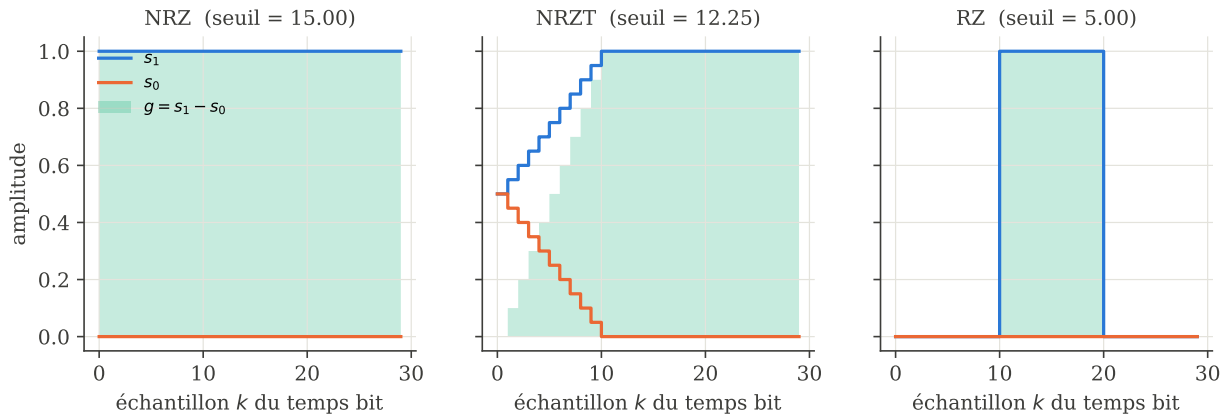


Figure 7: Formes de référence s_1 , s_0 et filtre $g = s_1 - s_0$ pour $N = 30$ et les amplitudes $(0, 1)$. Le seuil est exprimé en unités de la somme y .

pour s_1 et s_0 la **moyenne sur les deux valeurs possibles du bit précédent**. La partie qui dépend du bit précédent s'annule alors dans g : sur la rampe, $g[k] = (A_{\max} - A_{\min})k/t_1$, qui ne dépend que du bit courant. Le filtre n'est plus strictement optimal (le bit précédent crée une petite IES), mais la marge reste largement positive : sans bruit, toute séquence est décodée sans erreur.

16.4 Implémentation

Pour ne pas coder deux fois les formes d'onde, le récepteur demande s_1 et s_0 à l'émetteur, via la méthode `formeBit` extraite de `TransmetteurLogiqueAnalogique`. Le filtre et le seuil sont calculés une fois, à la construction :

```
public TransmetteurAnalogiqueLogique(String forme, int nbEch, float aMin,
    float aMax) {
    for (boolean bitPrecedent : new boolean[] {false, true}) { // moyenne (
        NRZT)
        float[] un = TransmetteurLogiqueAnalogique.formeBit(forme, nbEch, aMin,
            aMax, true, bitPrecedent);
        float[] zero = TransmetteurLogiqueAnalogique.formeBit(forme, nbEch,
            aMin, aMax, false, bitPrecedent);
        for (int k = 0; k < nbEch; k++) { s1[k] += un[k] / 2f; s0[k] += zero[k]
            / 2f; }
    }
    for (int k = 0; k < nbEch; k++) {
        filtre[k] = s1[k] - s0[k];
        seuil += filtre[k] * (s1[k] + s0[k]) / 2.0; // = (||s1||^2 - ||s0||^2)
            / 2
    }
}

public void emettre() {
    for (int i = 0; i < nbBits; i++) {
        double y = 0.0;
        for (int k = 0; k < nbEch; k++)
            y += informationRecue.iemeElement(i * nbEch + k) * filtre[k];
        informationEmise.add(y > seuil);
    }
}
```

Une nouvelle forme d'onde ajoutée à l'émetteur serait donc prise en charge par le récepteur sans modification. La commande unique ne change pas : le récepteur est choisi à partir de `-form`, `-nbEch` et `-ampl`,

déjà connus du simulateur.

16.5 TEB théorique

En sortie du filtre, le bruit est gaussien d'écart-type $\sigma_b \|g\|$, avec $\sigma_b^2 = P_s N / (2E_b/N_0)$ (étape 3). En l'absence d'IES :

$$\text{TEB} = Q\left(\frac{\|s_1 - s_0\|}{2\sigma_b}\right).$$

Forme	P_s	$\ s_1 - s_0\ ^2$	TEB (filtre adapté)
NRZ antipodal $(-A, A)$	A^2	$4A^2N$	$Q(\sqrt{2E_b/N_0})$
NRZ unipolaire $(0, A)$	$A^2/2$	A^2N	$Q(\sqrt{E_b/N_0})$
RZ unipolaire $(0, A)$	$A^2/6$	$A^2N/3$	$Q(\sqrt{E_b/N_0})$
RZ antipodal $(-A, A)$	A^2	$4A^2N/3$	$Q(\sqrt{2E_b/N_0/3})$

Pour le NRZT, et pour tous les cas avec échos, l'IES rend le TEB dépendant des bits voisins. Le script `courbesTEB.py` calcule alors une **théorie exacte** : il énumère toutes les suites possibles des bits voisins (le précédent pour la rampe, ceux qu'atteignent les échos), reconstruit la fenêtre reçue sans bruit avec les mêmes formes que l'émetteur, calcule P_s sur ce signal avec échos, puis moyenne $Q(\text{marge}/\sigma_y)$ sur toutes les suites. Le même calcul, avec $y = r[N/2]$ et $\sigma_y = \sigma_b$, donne la théorie du récepteur de l'étape 3.

17 Résultats

Toutes les courbes sont produites par `rapport/etape4/courbesTEB.py`, avec $N = 30$. Les points sont simulés par notre simulateur (jusqu'à 10^6 bits par point, en tranches de 200 000 bits avec des semences différentes) ; les traits sont la théorie exacte de la section 16.5. Les points à TEB nul ne sont pas représentés.

17.1 Filtre adapté : conformité à la théorie

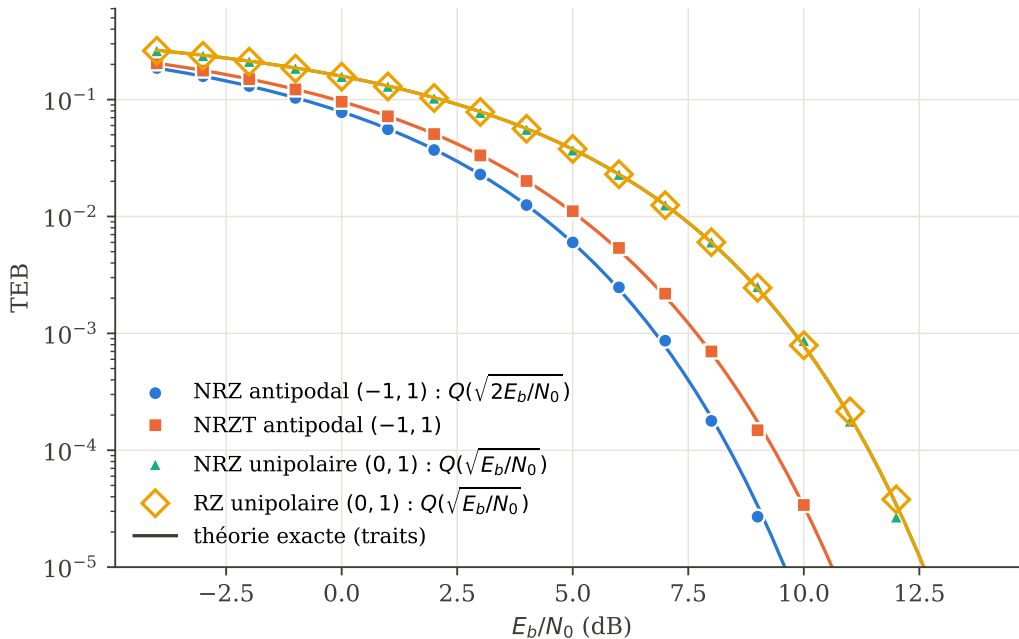


Figure 8: TEB du récepteur à filtre adapté (points) et théorie exacte (traits), canal gaussien sans écho.

Les points simulés se superposent à la théorie sur plus de quatre décades, jusqu'à $3 \cdot 10^{-5}$. L'écart médian entre simulation et théorie est de 0,8 %. L'écart relatif maximal (12 %) concerne des points à environ 170 erreurs, dont l'incertitude statistique relative est de l'ordre de $1/\sqrt{170} \approx 8\%$: il représente 1,5 écart-type. Sur les 313 points simulés pour ce rapport, aucun ne s'écarte de la théorie de plus de 2,5 écarts-types (loi binomiale), ce qui est conforme à des fluctuations purement statistiques.

Configuration	4 dB		8 dB	
	simulé	théorie	simulé	théorie
NRZ antipodal $(-1, 1)$	$1,25 \cdot 10^{-2}$	$1,25 \cdot 10^{-2}$	$1,78 \cdot 10^{-4}$	$1,91 \cdot 10^{-4}$
NRZT antipodal $(-1, 1)$	$2,01 \cdot 10^{-2}$	$2,01 \cdot 10^{-2}$	$7,0 \cdot 10^{-4}$	$6,8 \cdot 10^{-4}$
NRZ unipolaire $(0, 1)$	$5,60 \cdot 10^{-2}$	$5,65 \cdot 10^{-2}$	$6,09 \cdot 10^{-3}$	$6,00 \cdot 10^{-3}$
RZ unipolaire $(0, 1)$	$5,63 \cdot 10^{-2}$	$5,65 \cdot 10^{-2}$	$6,03 \cdot 10^{-3}$	$6,00 \cdot 10^{-3}$

Le NRZ unipolaire et le RZ unipolaire ont exactement la même courbe : avec le filtre adapté, seule compte l'énergie de la différence $s_1 - s_0$ rapportée à E_b , identique dans les deux cas (tableau de la section 16.5). Le NRZT perd environ 0,9 dB sur le NRZ : les rampes réduisent l'énergie utile de g et le bit précédent crée une légère IES.

17.2 Gain par rapport à l'étape 3

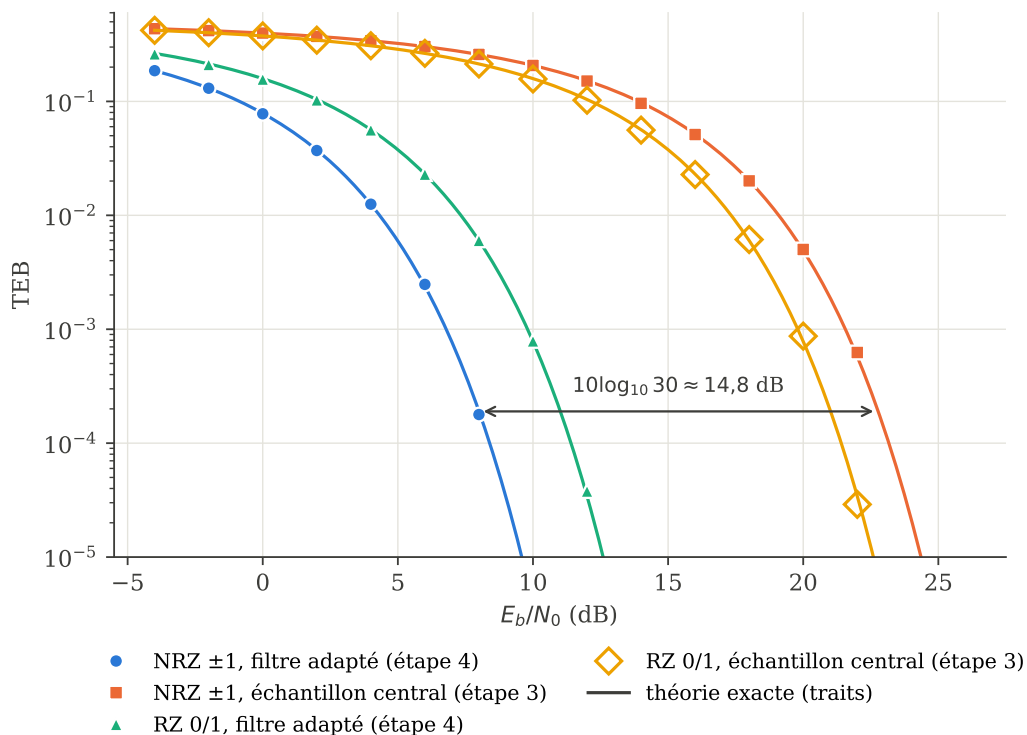


Figure 9: Récepteur de l'étape 3 (échantillon central, recompilé depuis le commit 82e4660) et récepteur à filtre adapté.

Le récepteur de l'étape 3 a été recompilé à partir du commit qui le contient encore et soumis aux mêmes balayages. Ses mesures suivent la théorie de la décision sur un échantillon (0,258 mesuré et théorique à 8 dB en NRZ). Le tableau donne le E_b/N_0 nécessaire pour atteindre un TEB de 10^{-3} (théorie exacte, confirmée par les points simulés) :

Forme	Étape 3 (échantillon central)	Étape 4 (filtre adapté)	Gain
NRZ antipodal $(-1, 1)$	21,6 dB	6,8 dB	14,8 dB
NRZT antipodal $(-1, 1)$	21,1 dB	7,7 dB	13,4 dB
RZ unipolaire $(0, 1)$	19,8 dB	9,8 dB	10,0 dB

Le gain vaut $10 \log_{10} M$, où M est le nombre d'échantillons qui portent l'information : $M = N = 30$ en NRZ (14,8 dB), mais seulement $M = N/3 = 10$ en RZ (10 dB), dont seul le tiers central distingue un 1 d'un 0. En NRZT, les rampes rendent une partie des échantillons moins utiles, d'où un gain intermédiaire. Le RZ, qui paraissait meilleur que le NRZ à l'étape 3 (artefact de la décision ponctuelle, déjà expliqué dans le rapport précédent), retrouve sa place théorique : 3 dB derrière le NRZ antipodal.

17.3 Trajets multiples sans bruit (étape 4a)

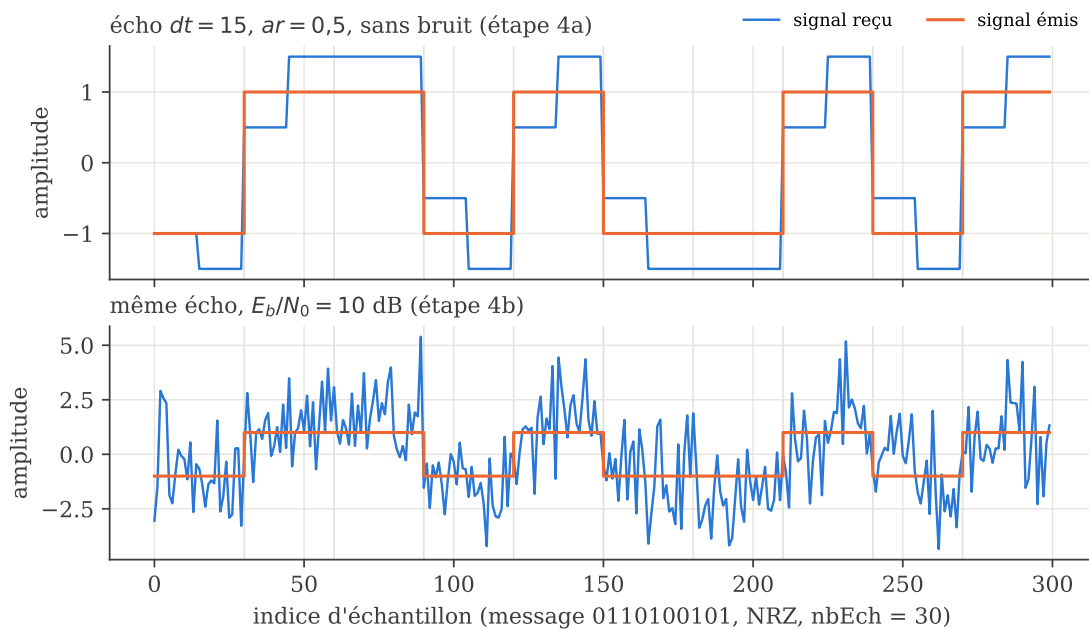


Figure 10: Signal NRZ $(-1, 1)$ émis et reçu avec un écho $dt = 15$, $ar = 0,5$ (sortie des classes du simulateur via ExportSignal).

La figure 10 montre l'effet d'un écho retardé d'un demi-bit : la première moitié de chaque bit reçoit l'écho du bit précédent, la seconde celui du bit courant. Quand deux bits consécutifs sont égaux, l'écho renforce le signal (1,5) ; quand ils diffèrent, il l'atténue (0,5) sur la première moitié du bit. Sans bruit, le TEB est soit nul, soit égal à $1/4$: toutes les erreurs viennent d'une même configuration, un bit 0 précédé d'un bit 1, dont l'écho fait franchir le seuil. Pour le NRZ unipolaire avec un écho $dt \leq N$, la sortie du filtre adapté (ici $g = 1$, seuil $N/2$) se calcule directement. Les dt premiers échantillons du bit courant b reçoivent l'écho du bit précédent p , les autres celui du bit courant :

$$y = \sum_{k < dt} (b + ar p) + \sum_{k \geq dt} (b + ar b) = Nb + ar (dt p + (N - dt) b).$$

Pour $b = 0$ et $p = 1$, $y = ar dt$ dépasse le seuil $N/2$ dès que $ar > N/(2dt)$. Cette configuration a une probabilité $1/4$:

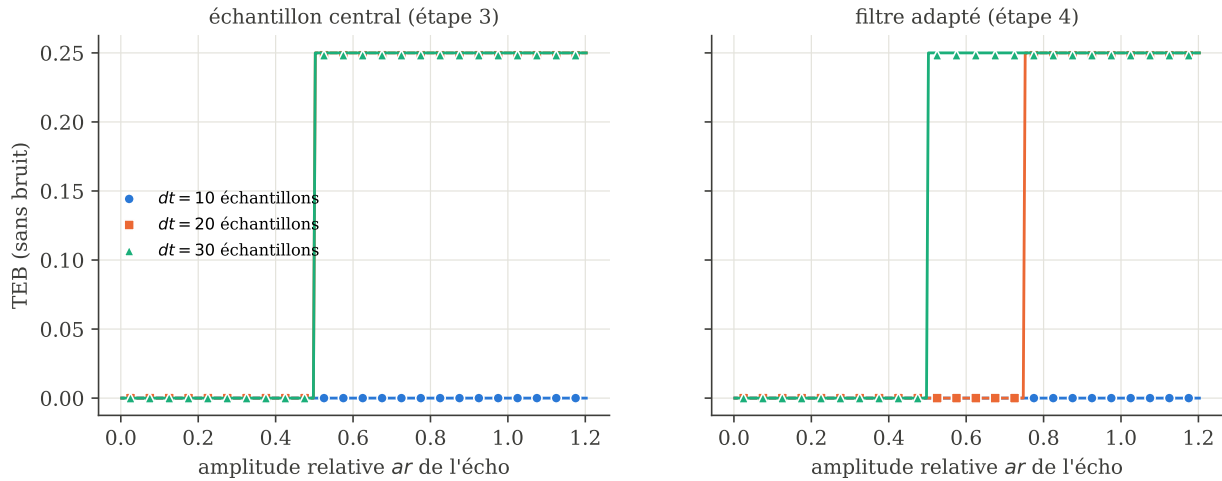


Figure 11: TEB sans bruit en fonction de l'amplitude de l'écho, NRZ unipolaire (0,1) (amplitudes par défaut), un seul trajet indirect, 20 000 bits par point. Pour $dt = 20$, la courbe de l'étape 3 est confondue avec celle de $dt = 30$.

dt	seuil d'erreur, filtre adapté	mesuré (filtre adapté)	seuil d'erreur, étape 3
10	$ar > 1,5$	aucune erreur jusqu'à 1,175 (fin du balayage)	jamais (l'échantillon central voit l'écho du bit courant)
20	$ar > 0,75$	premières erreurs à 0,775	$ar > 0,5$
30	$ar > 0,5$	premières erreurs à 0,525	$ar > 0,5$

Le filtre adapté résiste mieux à un écho qui ne recouvre qu'une partie du bit ($dt = 20$) : il moyenne l'écho du bit précédent avec la partie saine du bit, alors que l'échantillon central tombe en plein dans l'écho. Pour un écho d'un bit entier ($dt = N$), les deux récepteurs sont équivalents. En NRZ antipodal $(-1, 1)$, le seuil est à 0 et, pour tout retard, $y \cdot \text{signe}(b) \geq N(1 - ar)$: il faut $ar > 1$ pour une erreur, ce qui est physiquement exclu pour un trajet indirect ($ar < 1$) : c'est pourquoi le test JUnit des échos destructeurs utilise deux échos de 0,9 qui s'additionnent.

17.4 Trajets multiples et bruit (étape 4b)

Pour un écho d'un demi-bit en NRZ antipodal, le calcul de la section précédente donne deux cas équiprobables :

- bits précédent et courant égaux : l'écho renforce tout le bit, la marge est multipliée par $(1 + ar)$;
- bits différents : l'écho du bit précédent et celui du bit courant se compensent exactement, la marge est celle d'un canal sans écho.

Mais la puissance reçue augmente, $P_s = \frac{3}{4}(1 + ar)^2 + \frac{1}{4}(1 - ar)^2$, et le bruit, réglé sur $E_b = P_s T$ mesuré après les échos (fiche de la séance 4), augmente dans la même proportion :

$$\text{TEB} = \frac{1}{2} Q\left(\sqrt{2E_b/N_0/P_s}\right) + \frac{1}{2} Q\left((1 + ar)\sqrt{2E_b/N_0/P_s}\right).$$

ar	TEB à 8 dB (simulé)	TEB à 8 dB (théorie)	E_b/N_0 pour 10^{-3}	pénalité
0	$1,78 \cdot 10^{-4}$	$1,91 \cdot 10^{-4}$	6,8 dB	–
0,3	$7,2 \cdot 10^{-4}$	$6,7 \cdot 10^{-4}$	7,6 dB	0,9 dB
0,5	$1,88 \cdot 10^{-3}$	$1,83 \cdot 10^{-3}$	8,6 dB	1,8 dB
0,7	$4,2 \cdot 10^{-3}$	$4,1 \cdot 10^{-3}$	9,6 dB	2,8 dB

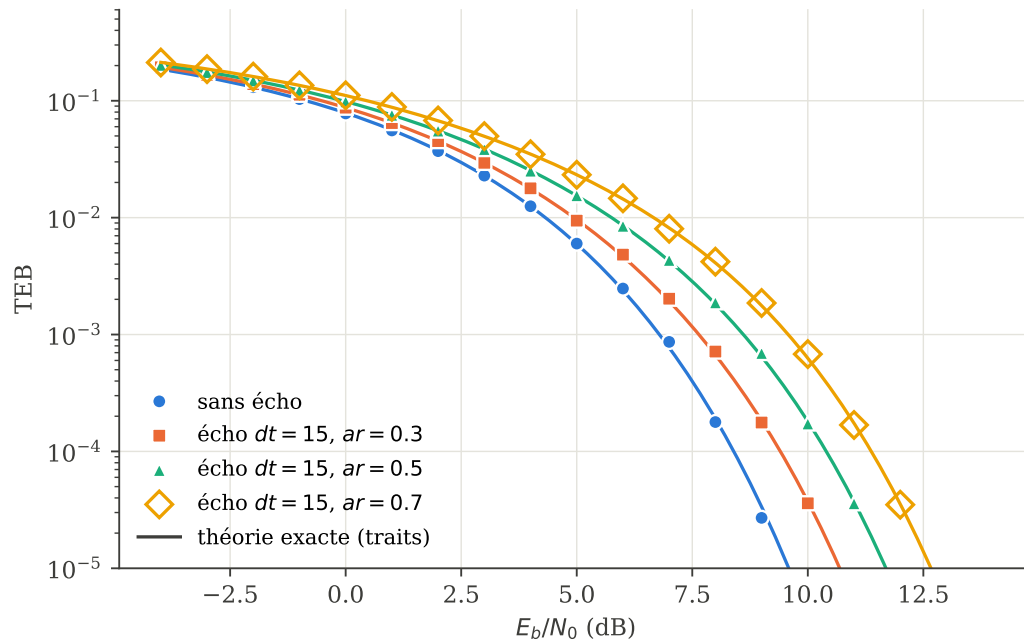


Figure 12: TEB en présence d'un écho $dt = 15$ (un demi-bit) et de bruit, NRZ antipodal $(-1, 1)$, filtre adapté.

La pénalité reste inférieure à l'augmentation de puissance $10\log_{10} P_s$ (1,4 ; 2,4 ; 3,4 dB) : la moitié du temps, l'écho renforce la décision, ce qui compense en partie.

18 Confrontation aux prévisions

18.1 Ce qui est conforme

- **Filtre adapté** : le TEB rejoint la borne optimale ($1,25 \cdot 10^{-2}$ à 4 dB en NRZ antipodal, simulé et théorique) et le gain prévu de 14,8 dB est mesuré. Les 3 dB entre antipodal et unipolaire sont retrouvés.
- **Trajets multiples sans bruit** : le TEB est nul tant que l'écho ne fait pas franchir le seuil, puis vaut exactement la probabilité des suites défavorables ($1/4$). Les seuils d'apparition des erreurs mesurés correspondent au calcul ($ar > N/(2dt)$).
- **Trajets multiples et bruit** : le TEB augmente avec ar à E_b/N_0 fixé, et la théorie exacte prédit chaque point.

18.2 Ce qui diffère de l'intuition

- L'écho d'un bit entier est bien le plus gênant, mais seulement pour les amplitudes unipolaires, dont le seuil est à mi-hauteur : il provoque des erreurs dès $ar > 0,5$. En NRZ antipodal, un écho *unique* d'amplitude $ar < 1$ ne crée jamais d'erreur sans bruit, quel que soit son retard ; il faut plusieurs échos qui s'additionnent.
- En 4b, une partie de la dégradation vient de la définition de E_b/N_0 : l'énergie de l'écho est comptée dans E_b alors qu'elle n'est utile que la moitié du temps.
- Le RZ est robuste aux échos courts : son impulsion est concentrée dans le tiers central, et l'écho de l'impulsion précédente n'atteint le tiers central du bit courant que si $dt > 2N/3$. Vérifié par simulation (RZ 0/1, 20 000 bits, sans bruit) : aucune erreur pour $dt \in \{10, 15, 20\}$ jusqu'à $ar = 1,1$, alors qu'à $dt = 30$ le TEB vaut 0,25 dès $ar = 0,7$.

19 Tests

19.1 Tests automatiques (runTests)

Le script couvre maintenant les étapes 1 à 4 dans des sections séparées ; chaque commentaire # Test n correspond au numéro affiché. Six tests concernent l'étape 4 :

```
Test 15 : Trajets indirects (1 trajet, sans bruit) ... OK
Test 16 : Trajets indirects (1 trajet, bruité) ... OK
Test 17 : Trajets indirects (3 trajets, bruité) ... OK
Test 26 : Erreur (ti sans couple) ... OK
Test 27 : Erreur (ti sans valeur ar) ... OK
Test 28 : Erreur (ti plus de 5 couples) ... OK
-----
RÉSULTAT : 28/28 OK
```

19.2 Tests unitaires JUnit

La suite passe de 75 à **96 tests** (22 ajoutés, 1 supprimé), tous réussis. Tests ajoutés :

- **canal à trajets multiples** (TransmetteurTest, préfixe TATM) : écho appliqué seulement après son retard ; bruit reproductible avec une semence ; signal vide ; *bruit identique à celui du canal gaussien sans trajet indirect* (vérifie le partage de la formule) ; rejet de tableaux incohérents, de plus de 5 trajets et d'un retard négatif ;

- **filtre adapté** (préfixe TAL) : en RZ, une forte perturbation hors du tiers central ne change pas la décision ; un échantillon central très faussé ne suffit plus à provoquer une erreur ; une séquence NRZT avec toutes les combinaisons de rampes est décodée sans erreur ;
- **simulateur** : `-ti` avec des échos destructeurs donne un TEB non nul ; `-ti` sans `-snrpb` n'ajoute pas de bruit ; un écho d'amplitude nulle donne exactement le même TEB que `-snrpb` seul ; `-ti` suivi d'autres options ; rejets des valeurs invalides ; *TEB conforme à la théorie* : NRZ antipodal à 4 dB, $1,25 \cdot 10^{-2}$ à $\pm 12\%$ sur 10^5 bits ;
- compléments de couverture ajoutés par L. Nanda sur l'émetteur (rampes NRZT, forme inconnue, signal vide) et le canal gaussien.

Le dernier test simulateur répond à la limite relevée à l'étape 3 : les tests vérifiaient que le simulateur affichait un TEB, pas sa valeur.

19.3 Limites identifiées

- **Semences liées.** Avec `-seed`, la source et le canal utilisent toujours la même semence, donc des suites pseudo-aléatoires liées. Utiliser `seed` et `seed + 1` garantirait leur indépendance. Les courbes restent conformes à la théorie, ce qui indique que l'effet est négligeable ici.
- **Mémoire.** `Information` stocke des `Float` (objets) : une simulation de 200 000 bits à 30 échantillons occupe environ 400 Mo. Les balayages sont donc découpés en tranches ; un stockage en `float[]` réduirait fortement l'empreinte. *Ce stockage a été réalisé à l'étape 5 (section 30).*
- **Pas de compensation des échos.** Le filtre adapté est optimal contre le bruit, pas contre l'IES. Un égaliseur est nécessaire pour annuler les trajets multiples.
- Le rapport de l'étape 3 prévoyait une option pour conserver la décision sur l'échantillon central. La commande unique n'en prévoyant pas, la comparaison est faite en recompilant la version de l'étape 3 depuis git (script de courbes).

20 Conclusion

Le canal à trajets multiples est intégré au simulateur avec l'option `-ti` de la commande unique, avec ou sans bruit. L'héritage de `TransmetteurAnalogiqueBruite` garantit que le bruit est calculé de la même façon dans les deux canaux, ce qui a supprimé une erreur de facteur N présente dans la première version.

Le récepteur à filtre adapté, annoncé comme prioritaire à la fin de l'étape 3, est en place. Une seule règle, la corrélation avec $g = s_1 - s_0$, sert aux trois formes d'onde. Le TEB mesuré atteint désormais la borne théorique, soit un gain de 14,8 dB en NRZ et 10 dB en RZ par rapport à l'étape 3. Toutes les mesures, avec ou sans échos, sont prédites par une théorie exacte de la chaîne, avec un écart médian inférieur à 1 %.

Le filtre adapté ne compense pas les échos : en présence de trajets multiples, le TEB reste dégradé (1,8 dB de pénalité pour un écho de 0,5 à un demi-bit) et, sans bruit, un écho fort impose un TEB de 1/4 en unipolaire. Deux pistes pour la suite : un égaliseur, qui estime et soustrait l'écho des bits déjà décidés, et le codage de canal de l'étape 5, qui pourra maintenant être évalué sur un récepteur optimal.

Partie III

Étape 5 – Codage de canal

21 Objectifs de l'étape 5

L'étape 5 insère un **codage de canal** dans la chaîne, pour diminuer le TEB de la transmission bruitée (TP5). Le travail demandé comporte trois points :

1. intégrer le codage : un codeur juste après la source, un décodeur juste avant la destination, activés par la nouvelle option `-codeur` (pas de codage par défaut) ;
2. mesurer l'intérêt du procédé : quelle amélioration du TEB selon la forme d'onde et le rapport signal à bruit ;
3. dire si le simulateur permet de mettre en évidence l'amélioration de la synchronisation émetteur/récepteur promise par le code.

L'étape s'appuie sur le récepteur à filtre adapté de la partie II : le gain du codage est mesuré sur une chaîne dont le TEB sans codage est déjà à la borne théorique.

22 Chaîne implémentée

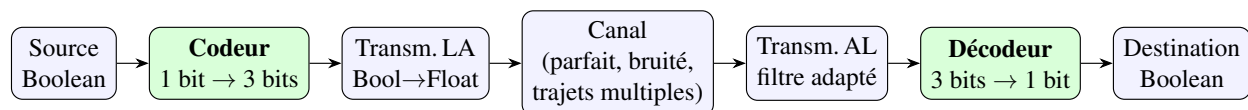


Figure 13: Chaîne de l'étape 5 avec l'option `-codeur`. En vert, les deux composants ajoutés. Le TEB est mesuré entre la source et la destination.

Sans `-codeur`, la chaîne est celle de l'étape 4. En simulation logique (aucune option analogique), le codeur et le décodeur entourent le transmetteur logique parfait.

23 Récapitulatif des modifications depuis l'étape 4

Élément	Modification
CodeurCanal	Nouvelle classe (Transmetteur<Boolean, Boolean>) : chaque bit reçu est émis sur trois bits, $0 \rightarrow 010$ et $1 \rightarrow 101$.
DecodeurCanal	Nouvelle classe : chaque paquet de trois bits redonne un bit, par choix du mot de code le plus proche. Une erreur par paquet est corrigée.
Simulateur	Option <code>-codeur</code> ; la chaîne se branche sur deux extrémités logiques (source et destination, ou codeur et décodeur) ; sondes logiques du message émis et du message décodé avec <code>-s</code> . Option <code>-decalage d</code> (extension hors commande unique) pour désynchroniser le récepteur.
TransmetteurAnalogiqueLogique	Décalage optionnel de la fenêtre de décision, pour simuler une désynchronisation (section 27) ; 0 par défaut, résultats inchangés.
VueDecalage	Nouvelle classe : fenêtre interactive de désynchronisation (curseur, tracé des fenêtres de décision sur le signal, TEB recalculé), ouverte avec <code>-s -decalage</code> .
InformationFlottante	Nouvelle classe (sous-classe d'Information<Float>) : le signal analogique est rangé dans un tableau de float au lieu d'objets Float, pour réduire la mémoire (section 30).
TransmetteurLogiqueAnalogique, TransmetteurAnalogiqueBruit	L'émetteur et le canal bruité (donc aussi le canal à trajets multiples) produisent une InformationFlottante. Leurs calculs sont inchangés.
Tests JUnit	29 tests ajoutés : 125 tests, tous réussis.
runTests	Section « Étape 5 » : 6 exécutions nominales, dont un test de mémoire et deux tests de désynchronisation à TEB exact, et 3 tests d'erreur (37 tests).
.gitignore	Les fichiers générés (bin/, docs/) ne sont plus versionnés.
genDeliverable, README	Archive etape5 ; description du contenu de l'archive et de l'option <code>-codeur</code> .
rapport/etape5/	Scripts courbesCodage.py (avec et sans codeur) et courbesDecalage.py (désynchronisation), avec théorie exacte ; outil OutilsDecalage.java (capture de la vue, paquets invalides).

Pour le codage de canal, aucune classe existante de la chaîne analogique n'a été modifiée. L'émetteur et le canal l'ont été ensuite, uniquement pour la façon de ranger le signal en mémoire : les TEB obtenus sont identiques au bit près.

24 Prévisions avant simulation

24.1 Paramètres pertinents

Paramètre	Option	Rôle
Codage de canal	<code>-codeur</code>	active le codeur et le décodeur
Rapport E_b/N_0 (dB)	<code>-snrpb</code>	s'applique à chaque bit <i>émis</i> , donc à chaque bit codé
Forme d'onde, amplitudes	<code>-form</code> , <code>-ampl</code>	fixent le TEB p d'un bit codé
Trajets indirects	<code>-ti</code>	erreurs liées aux bits voisins

24.2 Résultats attendus

- À **`-snrpb` égal**, le TEB doit fortement diminuer. Si chaque bit codé est faux avec la probabilité p , indépendamment des autres, le décodeur se trompe quand au moins deux bits du paquet sont faux : $TEB = 3p^2 - 2p^3 \approx 3p^2$. L'amélioration doit donc être d'autant plus forte que p est petit.
- À **énergie égale par bit d'information**, le résultat est moins évident : on émet trois bits pour un

bit utile, donc trois fois plus d'énergie. Chaque bit codé ne dispose que du tiers de l'énergie, soit $10\log_{10}3 \approx 4,8$ dB de moins. Le codage n'est rentable que s'il regagne plus que ces 4,8 dB.

- **Forme d'onde** : le décodeur ne voit que des bits ; le gain exprimé en dB ne devrait presque pas dépendre de la forme d'onde.
- **Trajets multiples** : les erreurs dues aux échos ne sont pas indépendantes d'un bit à l'autre. Le code, conçu pour une erreur isolée par paquet, pourrait être moins efficace.

25 Le codage retenu

25.1 Mots de code et décodage

Le codeur associe à chaque bit un mot de trois bits : le bit, son inverse, puis le bit.

Bit	Mot émis	000	001	010	011	100	101	110	111
0	010	0	1	0	0	1	1	0	1
1	101								

Les deux mots diffèrent sur leurs trois bits (distance de Hamming 3). Le décodeur choisit le mot le plus proche du paquet reçu (tableau de droite, celui du TP5) : un paquet qui contient **une** erreur est à distance 1 du bon mot et à distance 2 de l'autre, il est donc corrigé ; avec deux ou trois erreurs, le décodeur se trompe.

Ce code est un code à répétition dont le bit du milieu est inversé. Le mot le plus proche s'obtient donc par un **vote majoritaire** après avoir ré-inversé le bit du milieu, ce qui évite d'écrire les huit cas :

```
// CodeurCanal.emettre()
for (boolean bit : informationRecue) { // 0 -> 010, 1 -> 101
    informationEmise.add(bit);
    informationEmise.add(!bit);
    informationEmise.add(bit);
}

// DecodeurCanal.emettre()
for (int i = 0; i < nbPaquets; i++) {
    int debut = i * CodeurCanal.LONGUEUR_MOT;
    int votesPourUn = 0;
    if (informationRecue.iemeElement(debut)) votesPourUn++;
    if (!informationRecue.iemeElement(debut + 1)) votesPourUn++; // bit du
        milieu inversé
    if (informationRecue.iemeElement(debut + 2)) votesPourUn++;
    informationEmise.add(votesPourUn >= 2);
}
```

Un test JUnit vérifie que ce vote reproduit exactement les huit lignes de la table du TP5.

25.2 Lecture en termes d'automate

Le décodeur est un **transducteur déterministe** : il lit les bits trois par trois et, pour chacun des $2^3 = 8$ paquets possibles, il existe une et une seule sortie. Vu comme un automate, il part d'un état initial, compte les votes en lisant trois bits (au plus quatre états de comptage : 0, 1, 2 ou 3 votes pour « 1 »), émet un bit, puis revient à l'état initial. Le déterminisme a ici un intérêt pratique direct : le décodage ne demande ni retour en arrière ni mémoire au-delà du paquet en cours, et son coût est proportionnel à la longueur du message.

25.3 Intégration dans le simulateur

Le codeur et le décodeur sont des `Transmetteur<Boolean, Boolean>`, comme le transmetteur parfait de l'étape 1. Le simulateur définit deux « extrémités logiques », entre lesquelles il branche la chaîne logique ou analogique sans autre changement :

```
SourceInterface<Boolean> entree = source;
DestinationInterface<Boolean> sortie = destination;
if (codageCanal) { // option -codeur
    CodeurCanal codeur = new CodeurCanal();
    DecodeurCanal decodeur = new DecodeurCanal();
    source.connecter(codeur);
    decodeur.connecter(destination);
    entree = codeur;
    sortie = decodeur;
}
// ... entree.connecter(emetteur); ... recepteur.connecter(sortie);
```

L'option `-codeur` ne prend pas de paramètre, conformément à la commande unique. Elle se combine avec toutes les autres options (`-form`, `-snrpb`, `-ti`, `-s`).

25.4 TEB théorique après décodage

Le bruit est indépendant d'un bit codé à l'autre. Pour une suite de bits donnée, les erreurs sur les trois bits d'un paquet sont donc indépendantes, de probabilités p_1 , p_2 , p_3 , et

$$\text{TEB} = p_1 p_2 + p_1 p_3 + p_2 p_3 - 2 p_1 p_2 p_3 \quad (= 3p^2 - 2p^3 \text{ si } p_1 = p_2 = p_3 = p).$$

En NRZ et en RZ sans écho, p est le TEB sans codage de la partie II. En NRZT et avec des échos, les p_i dépendent des bits voisins. Le script `courbesCodage.py` énumère alors les suites de trois bits d'information, les code, reconstruit la fenêtre reçue de chaque bit du dernier paquet et moyenne la formule ci-dessus ; la puissance P_s qui règle le bruit est celle du signal *codé*.

26 Résultats

Les courbes sont produites par `rapport/etape5/courbesCodage.py`, avec $N = 30$. Les points sont simulés par notre simulateur (jusqu'à 600 000 bits d'information par point avec codeur, 10^6 sans) ; les traits sont la théorie exacte. Sur les 117 points simulés, aucun ne s'écarte de la théorie de plus de 2,5 écarts-types statistiques.

26.1 Amélioration du TEB à `-snrpb` égal

Configuration (simulé / théorie)	-snrpb 2		-snrpb 4	
	sans codeur	avec codeur	sans codeur	avec codeur
NRZ (-1, 1)	3,71 / $3,75 \cdot 10^{-2}$	4,05 / $4,11 \cdot 10^{-3}$	1,25 / $1,25 \cdot 10^{-2}$	4,75 / $4,65 \cdot 10^{-4}$
NRZT (-1, 1)	5,07 / $5,10 \cdot 10^{-2}$	7,98 / $7,66 \cdot 10^{-3}$	2,01 / $2,01 \cdot 10^{-2}$	1,07 / $1,21 \cdot 10^{-3}$
RZ (0, 1)	1,03 / $1,04 \cdot 10^{-1}$	2,94 / $3,02 \cdot 10^{-2}$	5,63 / $5,65 \cdot 10^{-2}$	8,95 / $9,21 \cdot 10^{-3}$

À `-snrpb` égal, le codage améliore nettement le TEB, et d'autant plus que le bruit est faible : le TEB est divisé par 9 à 2 dB et par 27 à 4 dB en NRZ antipodal, conformément à $3p^2$. Pour atteindre un TEB de 10^{-3} :

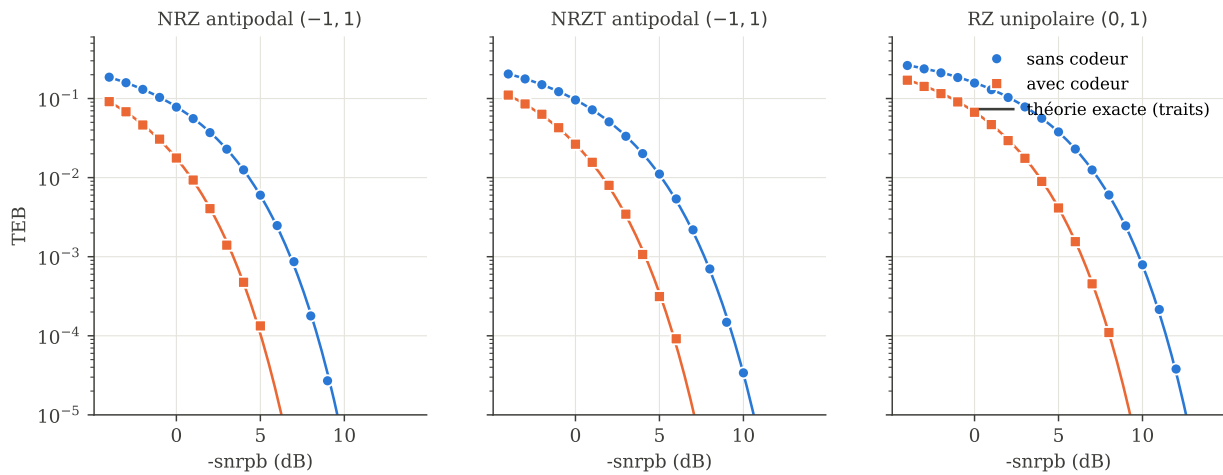


Figure 14: TEB avec et sans codeur pour les trois formes d'onde, en fonction de $-\text{snrpb}$ (rapport E_b/N_0 de chaque bit émis).

Forme	$-\text{snrpb}$ sans codeur	$-\text{snrpb}$ avec codeur	Gain
NRZ antipodal $(-1, 1)$	6,8 dB	3,4 dB	3,4 dB
NRZT antipodal $(-1, 1)$	7,7 dB	4,2 dB	3,5 dB
RZ unipolaire $(0, 1)$	9,8 dB	6,4 dB	3,4 dB

Le gain est pratiquement le même pour les trois formes d'onde : le décodeur ne traite que des bits, la forme d'onde ne fait que fixer p . Le léger écart du NRZT vient de la statistique du signal codé, qui change de valeur plus souvent (voir section 26.3) : les rampes sont plus fréquentes et la puissance P_s un peu plus faible.

26.2 Comparaison à énergie égale par bit d'information

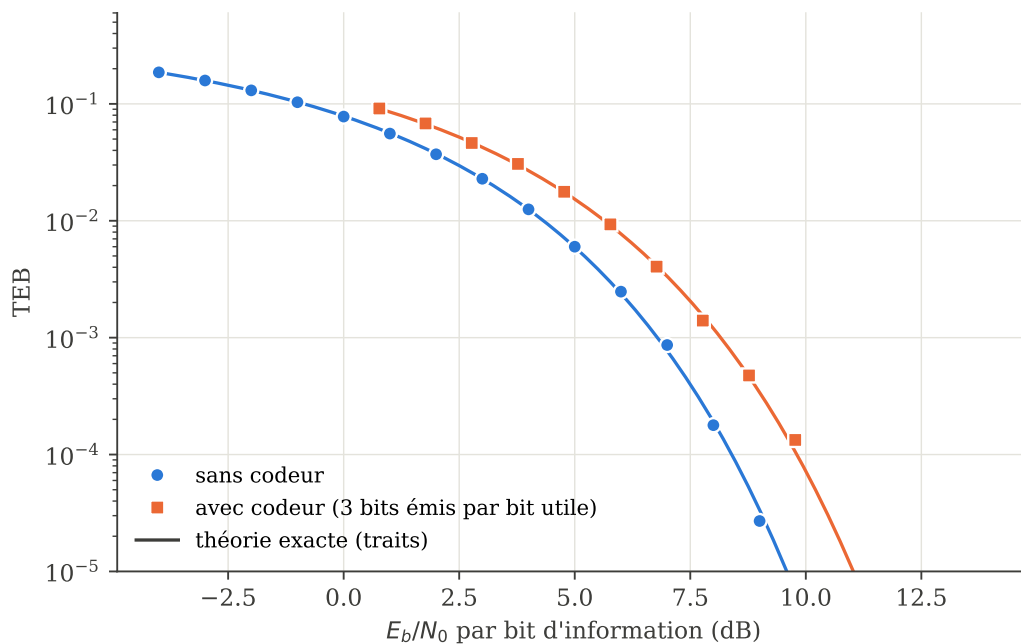


Figure 15: NRZ antipodal : TEB en fonction de E_b/N_0 par bit d'information. Avec le codeur, l'énergie d'un bit utile est celle de ses trois bits codés ; la courbe est celle de la figure 14, décalée de $10\log_{10} 3 = 4,8$ dB.

La comparaison précédente avantage le codage : avec `-codeur`, le simulateur émet trois bits par bit utile, chacun avec la même énergie. La transmission consomme donc **trois fois plus d'énergie** et, à débit d'émission constant, dure trois fois plus longtemps. Si l'on raisonne à énergie égale par bit d'information, la courbe codée se décale de 4,8 dB vers la droite (figure 15) et passe *au-dessus* de la courbe sans codage :

E_b/N_0 par bit utile pour 10^{-3}	sans codeur	avec codeur	bilan
NRZ antipodal $(-1, 1)$	6,8 dB	8,2 dB	perte de 1,4 dB
NRZT antipodal $(-1, 1)$	7,7 dB	8,9 dB	perte de 1,3 dB
RZ unipolaire $(0, 1)$	9,8 dB	11,2 dB	perte de 1,4 dB

Le code regagne 3,4 dB par la correction d'erreur, mais en dépense 4,8 par sa redondance. Ce résultat est connu : le gain de codage asymptotique d'un code de rendement R qui corrige t erreurs, décodé par décisions fermes, vaut $10\log_{10}(R(t+1))$ [2], soit ici $10\log_{10}(2/3) = -1,8$ dB. Nos mesures ($-1,3$ à $-1,4$ dB entre 10^{-2} et 10^{-5}) tendent vers cette valeur. Un décodage souple, qui additionnerait les trois sorties du filtre adapté avant de décider, ramènerait exactement aux performances sans codage : un code à répétition n'apporte pas de gain de codage, il échange du débit contre de la fiabilité.

Réponse à la question 2 du TP5. Le codage améliore le TEB à puissance d'émission et rapport signal à bruit par bit émis constants (gain de 3,4 dB, quelle que soit la forme d'onde), au prix d'un débit utile divisé par trois. Il ne constitue pas une amélioration à énergie constante par bit d'information. Son intérêt pratique est donc de fiabiliser une liaison dont on ne peut pas augmenter la puissance, quand on accepte de réduire le débit.

26.3 Codage et trajets multiples

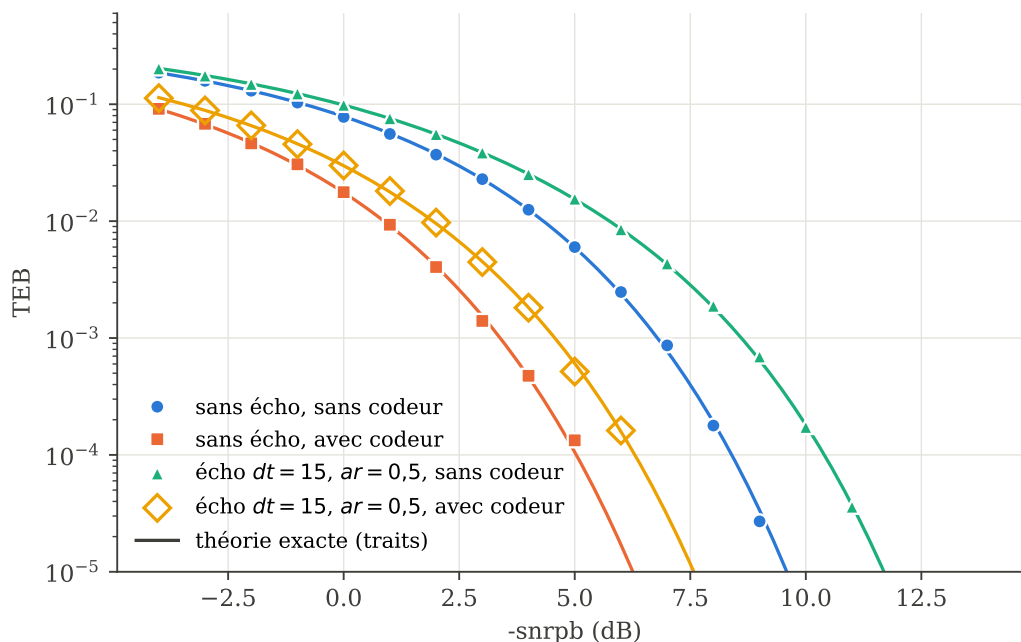


Figure 16: NRZ antipodal $(-1, 1)$, avec et sans écho ($dt = 15$, $ar = 0,5$), avec et sans codeur.

Avec bruit (étape 4b). Le codage reste efficace en présence d'un écho, et la pénalité due à l'écho est même plus faible avec le codeur :

$-snrpb$ pour 10^{-3}	sans écho	écho $dt = 15$, $ar = 0,5$	pénalité de l'écho
sans codeur	6,8 dB	8,6 dB	1,8 dB
avec codeur	3,4 dB	4,6 dB	1,2 dB

L'explication tient à la forme du signal codé. Sans codage, deux bits consécutifs diffèrent une fois sur deux. Avec le code, les trois bits d'un mot alternent toujours, et seul le passage d'un mot au suivant peut conserver la valeur : deux bits consécutifs diffèrent **5 fois sur 6**. Or, pour un écho d'un demi-bit, la partie II a montré que l'écho n'aide la décision que si les bits sont égaux, et que le bruit est réglé sur la puissance reçue P_s . Avec le signal codé, $P_s = \frac{1}{6}(1+ar)^2 + \frac{5}{6} \cdot \frac{1}{2}[(1-ar)^2 + (1+ar)^2] = 1,42$ pour $ar = 0,5$, contre 1,75 sans codage : le bruit ajouté pour un même $-snr_{pb}$ est plus faible.

Sans bruit (étape 4a). En NRZ unipolaire (0, 1), où un écho fort fait franchir le seuil (partie II), le codage **n'apporte rien** :

dt	ar	sans codeur (simulé)	avec codeur (simulé)	théorie (les deux)
30	0,4	0	0	0
30	0,6	0,248	0,248	0,25
20	0,6	0	0	0
20	0,9	0,248	0,248	0,25
10	0,9	0	0	0

Sans bruit, un bit 0 est faux chaque fois qu'il suit un bit 1 dont l'écho dépasse le seuil. Dans le mot 010 qui suit un mot 101, le premier et le troisième bit sont deux 0 précédés d'un 1 : le paquet contient **deux** erreurs, que le décodeur ne peut pas corriger. Ce cas se produit quand un bit d'information 0 suit un bit 1, soit une fois sur quatre : le TEB vaut 1/4, comme sans codage. Le code corrige des erreurs isolées et aléatoires ; il est sans effet contre des erreurs systématiques, liées au signal lui-même. Contre les échos, il faut un égaliseur.

26.4 Synchronisation émetteur/récepteur

Dans le simulateur, le récepteur connaît exactement le début de chaque bit : la synchronisation est parfaite par construction. La question 3 du TP5, qui porte sur l'amélioration de la synchronisation, demande donc de pouvoir simuler une désynchronisation. Elle fait l'objet de la section suivante.

27 Désynchronisation du récepteur (question 3 du TP5)

La question 3 du TP5 demande si le simulateur permet de mettre en évidence l'amélioration de la synchronisation émetteur/récepteur promise par le code. Pour y répondre, nous avons rendu la fenêtre de décision du récepteur réglable : on peut ainsi **simuler une désynchronisation** et observer son effet sur les signaux et sur le TEB.

27.1 Modèle et réalisation

Par défaut, l'émetteur et le récepteur partagent la même base de temps : le bit i occupe exactement les échantillons iN à $(i+1)N - 1$, et le récepteur intègre exactement cette fenêtre. La synchronisation est parfaite. Nous modélisons une **erreur de synchronisation constante** par un décalage d de la fenêtre de décision :

$$\text{fenêtre du bit } i = [iN + d, iN + d + N[, \quad |d| < N,$$

$d > 0$ correspondant à un récepteur en retard sur l'émetteur et $d < 0$ à un récepteur en avance. La fenêtre mord donc sur le bit suivant ou sur le bit précédent. Avant le début et après la fin du signal, rien n'est reçu (échantillons nuls).

```
// TransmetteurAnalogiqueLogique.emettre()
// filtre adapté sur la fenêtre décalée
for (int i = 0; i < nbBits; i++) {
```

```

int debutFenetre = i * nbEch + decalage;
double y = 0.0;
for (int k = 0; k < nbEch; k++) {
    y += echantillonRecu(debutFenetre + k) * filtre[k]; // 0 hors du signal
}
informationEmise.add(y > seuil);
}

```

- **Option -decalage d** : le simulateur crée un récepteur décalé de d échantillons, et le TEB affiché tient compte de la désynchronisation. Cette option n'existe pas dans la commande unique : c'est une extension, sans effet quand elle est absente. La borne $|d| < N$ est vérifiée une fois toutes les options lues, puisque $-nbEch$ peut venir après.
- **Fenêtre interactive** : avec `-s -decalage d`, la fenêtre `VueDecalage` s'ouvre après la simulation. Un curseur, ou la molette, fait varier d de $-(N-1)$ à $N-1$; le signal reçu est de nouveau décidé (et décodé avec `-codeur`) à chaque mouvement, le tracé montre les fenêtres de décision sur le signal et le TEB du message complet est recalculé (figure 17).
- Sans décalage, les résultats sont identiques au bit près à ceux de la version précédente.

27.2 Effet sur les signaux

Avec $d = 10$ (un tiers de bit), chaque fenêtre contient deux tiers du bit courant et un tiers du bit suivant : la décision reste juste. Avec $d = 20$, chaque fenêtre contient davantage du bit suivant que du bit courant : le récepteur décide le bit suivant, et toutes les fenêtres où le bit change sont fausses.

27.3 Effet sur le TEB sans bruit

Sans bruit, le TEB est nul tant que la fenêtre contient davantage du bit courant que de son voisin, puis saute brusquement :

Forme	décalages sans erreur	TEB au-delà, sans codeur	TEB au-delà, avec codeur
NRZ antipodal $(-1, 1)$	$-14 \leq d \leq 14$	0,5	1
NRZT antipodal $(-1, 1)$	$-12 \leq d \leq 17$	0,5	1
RZ unipolaire $(0, 1)$	$-4 \leq d \leq 4$	0,5	0,5, puis 1 si $ d \geq 26$

- **NRZ** : la sortie du filtre vaut $2[(N-|d|)s_i + |d|s_{\text{voisin}}]$. Tant que $|d| < N/2$, le bit courant l'emporte. Au-delà, le récepteur décide la valeur du voisin, et se trompe chaque fois que les deux bits diffèrent, soit une fois sur deux.
- **NRZT** : la tolérance est dissymétrique. Un récepteur en retard ($d > 0$) saute la rampe du début du bit, qui dépend du bit précédent, et n'atteint que la rampe du bit suivant, qui part du niveau du bit courant : il tolère jusqu'à 17 échantillons. En avance, il intègre la fin du bit précédent et ne tolère que 12 échantillons.
- **RZ** : l'impulsion n'occupe que le tiers central du bit. Dès que la fenêtre se décale de la moitié de cette impulsion ou plus ($N/6 = 5$ échantillons), elle ne voit plus assez d'impulsion pour décider un 1 : tous les bits 1 sont perdus. C'est la forme d'onde la plus sensible à la désynchronisation. Pour $|d| \geq 26$, la fenêtre voit au contraire plus de la moitié de l'impulsion du bit *voisin*, et décide ce bit.

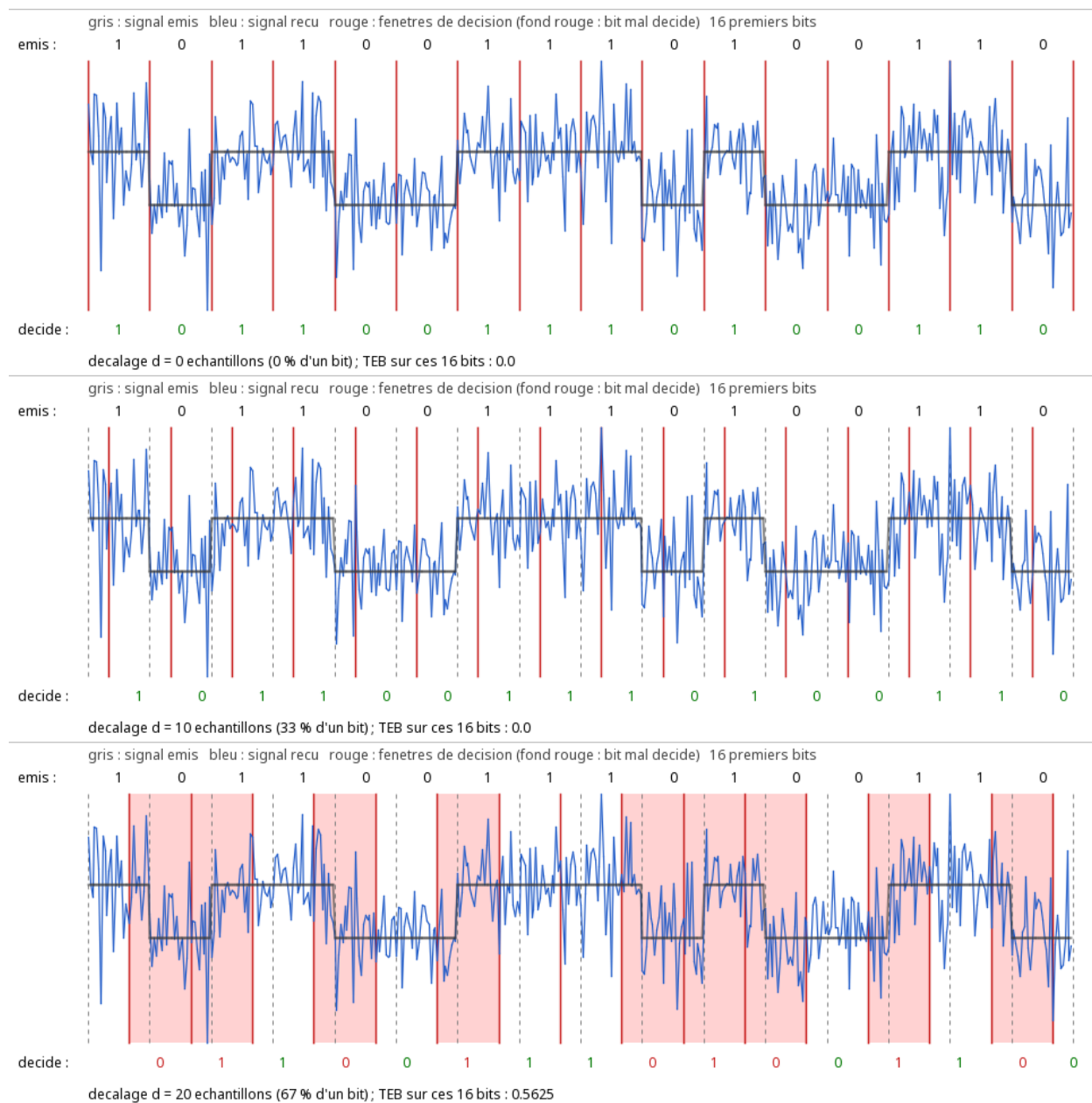


Figure 17: Tracé de la fenêtre interactive (NRZ antipodal, $E_b/N_0 = 10$ dB, 16 premiers bits) pour $d = 0$, 10 et 20 échantillons. Gris : signal émis ; bleu : signal reçu ; pointillés : limites des bits ; traits rouges : fenêtres de décision ; fond rouge : bit mal décidé.

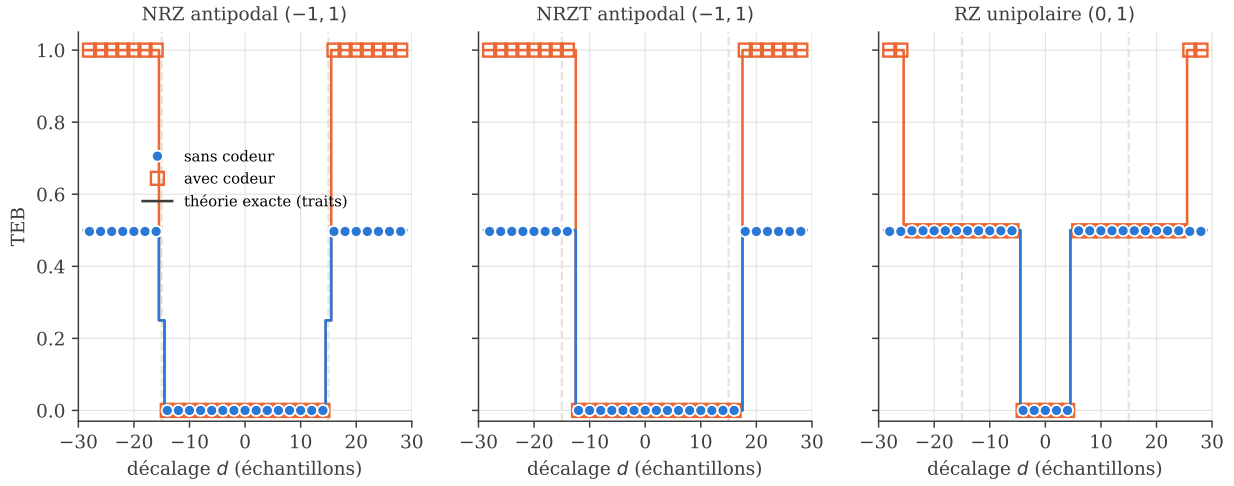


Figure 18: TEB sans bruit en fonction du décalage d ($N = 30$), avec et sans codeur. Points : simulations (20 000 bits, un point sur deux tracé) ; traits : théorie exacte. Pointillés : $|d| = N/2$.

- **Avec le codeur**, la tolérance est la même, mais au-delà le TEB vaut 1 en NRZ et NRZT : *tout le message est inversé*. Le récepteur lit, pour le paquet $(b, \bar{b}, b)$, les bits $(\bar{b}, b, b')$; après ré-inversion du bit du milieu, le vote majoritaire donne toujours $\bar{b}$. En RZ, pour $5 \leq |d| \leq 25$, toutes les décisions valent 0 et le décodeur rend 0 pour chaque paquet : le TEB reste de 0,5 ; pour $|d| \geq 26$, le récepteur lit le bit voisin et le message est de nouveau inversé.

27.4 Effet sur le TEB avec bruit

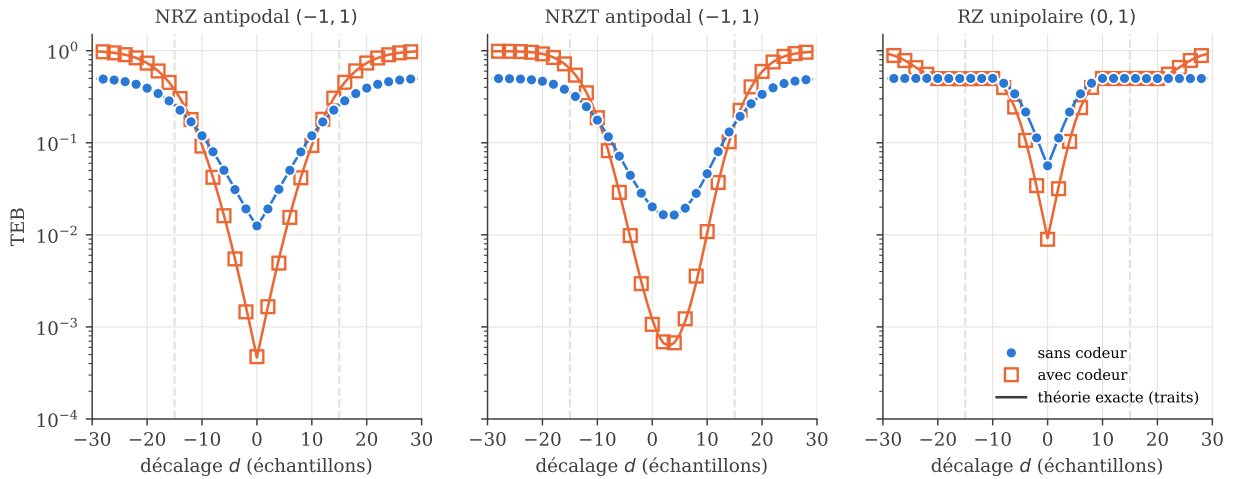


Figure 19: TEB en fonction du décalage d à $E_b/N_0 = 4$ dB, avec et sans codeur. Points : simulations ; traits : théorie exacte.

Valeurs mesurées et théoriques en NRZ antipodal (TEB) :

d (échantillons)	-10	-4	0	4	10	14	20
sans codeur, simulé	0,119	0,031	0,0125	0,0313	0,119	0,227	0,392
sans codeur, théorie	0,12	0,0313	0,0125	0,0313	0,12	0,227	0,393
avec codeur, simulé	0,0921	0,00548	0,000475	0,00493	0,0934	0,306	0,735
avec codeur, théorie	0,0939	0,00549	0,000465	0,00549	0,0939	0,306	0,735

- Avec du bruit, il n’y a plus de tolérance : le TEB augmente dès le premier échantillon de décalage, car la partie du voisin réduit la marge de décision. En NRZ, un décalage d’un sixième de bit ($d = 5$) triple déjà le TEB.
- **Le codeur reste utile pour les petits décalages**, mais devient nuisible au-delà d’environ $|d| = 12$ en NRZ : son TEB tend vers 1 alors que celui sans codeur tend vers 0,5. En NRZT, le croisement a lieu à $d = 16$ en retard mais dès $d = -10$ en avance ; en RZ, les deux TEB valent 0,5 dès $|d| \geq 10$.
- L’effet est rapide : en NRZ, une erreur de synchronisation d’un tiers de bit multiplie le TEB par 10 sans codeur et par 200 avec codeur.

27.5 Le code permet de détecter une désynchronisation

Le code ne protège donc pas contre une erreur de synchronisation. Sa structure permet en revanche de la **détecter**. Quand le récepteur est synchronisé, les paquets reçus sont des mots de code (010 ou 101), sauf erreur due au bruit. Quand il est décalé de plus d’un demi-bit, il lit $(\bar{b}, b, b')$, qui n’est un mot de code que si $b' \neq b$: un paquet sur deux n’en est plus un.

NRZ antipodal, avec codeur, d	0	5	10	14	16	20
sans bruit : paquets invalides	0,0 %	0,0 %	0,0 %	0,0 %	50,3 %	50,3 %
sans bruit : TEB	0	0	0	0	1	1
4 dB : paquets invalides	3,8 %	16,5 %	46,8 %	71,2 %	77,1 %	73,8 %
4 dB : TEB	0,000333	0,00973	0,0947	0,307	0,451	0,733

Un récepteur qui surveille la proportion de paquets invalides peut donc repérer une désynchronisation, avant même que le TEB ne s’effondre, et se recalcr (par exemple en essayant les trois positions possibles du début des paquets et en gardant celle qui donne le moins de paquets invalides). C’est un premier apport concret du code à la synchronisation.

27.6 Réponse à la question 3

En partie. Le simulateur permet désormais de :

- **simuler une désynchronisation** constante et observer son effet sur les signaux (fenêtre interactive) et sur le TEB (option `-decalage`) ;
- vérifier que le code **ne protège pas** contre une erreur de synchronisation : il la rend même catastrophique au-delà d’un demi-bit (message inversé) ;
- montrer que le code permet de la **détecter** (paquets invalides), et vérifier la propriété annoncée par le TP : jamais plus de deux bits consécutifs identiques dans le signal codé (test JUnit).

L’amélioration promise par le TP concerne cependant la **recupération du rythme** : un récepteur réel recalc en permanence son horloge sur les transitions du signal reçu, et une longue suite de bits identiques, sans transition, le laisse dériver. Notre modèle utilise un décalage *constant* et un récepteur qui ne se recalc jamais : il ne peut pas montrer ce gain. Il faudrait pour cela modéliser une horloge de réception légèrement différente de celle de l’émission (un décalage qui grandit au fil des bits) et un récepteur qui se recalc sur chaque transition détectée. Sans codage, les longues suites de bits identiques laisseraient la dérive s’accumuler ; avec le code, une transition au moins tous les deux bits la corrigerait.

28 Confrontation aux prévisions

28.1 Ce qui est conforme

- Le TEB après décodage suit $3p^2 - 2p^3 : 4,05 \cdot 10^{-3}$ mesuré à 2 dB en NRZ antipodal pour $4,11 \cdot 10^{-3}$ attendu.
- Le gain à $-snr_{pb}$ égal (3,4 dB) ne dépend presque pas de la forme d'onde.
- À énergie égale par bit d'information, le code perd 1,4 dB : il ne regagne pas les 4,8 dB de sa redondance.
- Contre les échos sans bruit, le code est inefficace, comme on pouvait le craindre pour des erreurs non indépendantes.
- Les 528 points de TEB simulés en fonction du décalage de la fenêtre de décision concordent avec la théorie exacte, à moins de 2,5 écarts-types statistiques.

28.2 Ce qui diffère de l'intuition

- Avec du bruit, le codage est *plus* efficace en présence d'un écho qu'en son absence (4,0 dB de gain au lieu de 3,4). Nous attendions l'inverse. L'effet vient de l'alternance imposée par le code, qui réduit la puissance reçue, donc le bruit réglé sur elle.
- L'échec du code contre les échos forts n'est pas partiel mais total : le TEB est exactement le même avec et sans codeur.
- Le code, présenté comme une aide à la synchronisation, rend une erreur de synchronisation de plus d'un demi-bit catastrophique : le message entier est inversé (TEB de 1 au lieu de 0,5). Son apport est ailleurs : il permet de *détecter* la désynchronisation.
- La tolérance du NRZT à la désynchronisation n'est pas symétrique : un récepteur en retard tolère 17 échantillons, un récepteur en avance seulement 12.

29 Tests

29.1 Tests automatiques (runTests)

Neuf tests concernent l'étape 5 ; les tests d'erreur ont été renumérotés.

```
Test 18 : Codage de canal (logique) ... OK
Test 19 : Codage de canal (analogique bruité) ... OK
Test 20 : Codage de canal (trajets indirects, bruité) ... OK
Test 21 : Mémoire (100 000 bits codés, tas Java de 256 Mo) ... OK
Test 22 : Désynchronisation faible (décalage 14 < nbEch/2) : TEB nul ... OK
Test 23 : Désynchronisation forte avec codeur (décalage 16) : message inversé
... OK
Test 35 : Erreur (codeur avec paramètre) ... OK
Test 36 : Erreur (decalage non entier) ... OK
Test 37 : Erreur (decalage >= nbEch) ... OK
-----
RÉSULTAT : 37/37 OK
```

Les tests 22 et 23 comparent le TEB affiché *exactement* à la valeur attendue (0,0 et 1,0), ce que permettent ces simulations sans bruit. Le test 21 vérifie le gain de mémoire de la section 30 : il simule 100 000 bits codés (9 millions d'échantillons) en limitant la mémoire de la machine virtuelle Java à 256 Mo (variable

d'environnement `JAVA_TOOL_OPTIONS=-Xmx256m`). Avec l'ancien stockage, cette simulation s'arrête sur une `OutOfMemoryError` ; avec le nouveau, elle réussit, et réussit encore avec 128 Mo.

29.2 Tests unitaires JUnit

La suite passe de 96 à **125 tests**, tous réussis. Tests ajoutés :

- **codeur** : $0 \rightarrow 010$ et $1 \rightarrow 101$; jamais trois bits consécutifs identiques dans le signal codé ;
- **décodeur** : les huit lignes de la table du TP5 ; une erreur sur le premier, le deuxième ou le troisième bit d'un paquet est corrigée ; deux erreurs dans un paquet donnent le mauvais bit (limite du code) ; message vide et paquet incomplet en fin de message ;
- **simulateur** : TEB nul avec `-codeur` en logique et sans bruit pour les trois formes d'onde ; *TEB conforme à la théorie* avec et sans codeur ($3,75 \cdot 10^{-2}$ et $4,1 \cdot 10^{-3}$ à 2 dB sur 60 000 bits) ; combinaison avec `-ti` et `-snrpb` ; refus de `-codeur 3` ;
- **stockage en float[]** (`InformationTest`) : ajout de valeurs et agrandissement du tableau ; égalité avec une `Information<Float>` classique, dans les deux sens ; parcours « for each » (utilisé par les sondes), affichage et modification identiques ; refus d'un rang hors des valeurs rangées et de la valeur `null` ; utilisation effective par l'émetteur et le canal ;
- **désynchronisation** : décisions exactes du récepteur décalé (un décalage de moins d'un demi-bit ne change rien, +6 sur 10 échantillons fait lire le bit suivant, -6 le bit précédent) ; refus d'un décalage d'au moins un bit ; option `-decalage` (valeurs refusées quel que soit l'ordre des options, $d = 0$ identique à l'absence d'option, TEB exacts sans bruit, message inversé avec le codeur, TEB conforme à la théorie avec bruit : 0,1200 à 4 dB pour $d = 10$) ; calculs et tracé de la fenêtre interactive, vérifiés sans écran.

30 Performances du simulateur

Avec le codeur, le signal est trois fois plus long, et les simulations longues atteignaient les limites de mémoire d'un ordinateur portable. Nous avons donc modifié la façon dont le signal analogique est rangé en mémoire.

30.1 Constat

Simulation de 200 000 bits, $N = 30$, avant modification	mémoire	durée
NRZ bruité, sans codeur	414 Mo	0,82 s
NRZ bruité, avec <code>-codeur</code>	1 074 Mo	2,07 s

(Mémoire maximale occupée par le processus et durée totale ; médiane de trois exécutions, machine virtuelle limitée à 1,1 Go.)

La cause est le stockage du signal analogique. La classe `Information` range ses éléments dans une `ArrayList<Float>` : chaque échantillon est un *objet* `Float` (16 octets) plus une référence (4 octets), soit 20 octets au lieu des 4 d'un `float`. Le signal existe de plus en deux exemplaires (sortie de l'émetteur, sortie du canal). Pour 200 000 bits codés, cela représente 18 millions d'échantillons, soit environ 360 Mo par exemplaire. La création de ces millions d'objets explique aussi l'essentiel du temps de calcul.

30.2 Modification réalisée : le signal rangé dans des tableaux de float

Nous avons ajouté au paquetage `information` une sous-classe `InformationFlottante` d'`Information<Float>`, qui range ses valeurs dans un tableau de `float` :

```
public class InformationFlottante extends Information<Float> {
    private float[] valeurs; // seules les cases 0 .. taille-1 servent
    private int taille;

    public float valeur(int i) {
        return valeurs[Objects.checkIndex(i, taille)];
    }
    public void ajouter(float valeur) {
        // (le tableau est agrandi si besoin)
        valeurs[taille++] = valeur;
    }
    @Override public int nbElements() { return taille; }
    @Override public Float iemeElement(int i) { return valeur(i); }
    // setIemeElement, add et iterator : redéfinies de la même façon
}

// TransmetteurLogiqueAnalogique.emettre() : taille du signal connue d'avance
InformationFlottante signal = new InformationFlottante(nbBits * nbEch);
signal.ajouter(formeBit(forme, nbEch, aMin, aMax, bit, bitPrecedent));
```

- **Où** : l'émetteur (`TransmetteurLogiqueAnalogique`) et le canal bruité (`TransmetteurAnalogiqueBruit`) créent désormais une `InformationFlottante`, dimensionnée d'avance. Le canal à trajets multiples, qui hérite du canal bruité, en profite sans modification. Le canal parfait transmet le même objet.
- **Pourquoi une sous-classe** : le reste du code (récepteur, sondes, simulateur, tests) manipule une `Information<Float>` uniquement par ses méthodes `nbElements`, `iemeElement` et le parcours « for each ». En les redéfinissant, la nouvelle classe s'utilise partout sans autre changement. L'itérateur doit lui aussi être redéfini, car celui d'`Information` lit directement sa liste interne ; or les sondes parcourent le signal avec « for each ».
- **Seule différence** : la valeur `null`, qu'un `float` ne peut pas représenter, est refusée. Le simulateur n'en produit jamais.

Vérification. Les calculs sont inchangés : seule la façon de ranger les valeurs change. Nous avons comparé l'ancienne et la nouvelle version sur 89 configurations (trois formes d'onde, deux jeux d'amplitudes, avec et sans bruit, échos, codeur, deux semences, messages fixes) : les TEB sont **identiques au bit près**. Tous les tests `JUnit` et `runTests` passent. Les courbes de ce rapport, calculées avant la modification, restent donc valables.

200 000 bits, $N = 30$	avant	après	gain
NRZ bruité, sans codeur	414 Mo, 0,82 s	152 Mo, 0,40 s	mémoire ÷ 2,7, durée ÷ 2,0
NRZ bruité, avec -codeur	1 074 Mo, 2,07 s	294 Mo, 1,01 s	mémoire ÷ 3,7, durée ÷ 2,0
NRZT, écho, avec -codeur	1 076 Mo, 2,12 s	298 Mo, 1,08 s	mémoire ÷ 3,6, durée ÷ 2,0

Le gain de mémoire est le plus fort pour les signaux longs, où le signal pèse davantage que la mémoire fixe de la machine virtuelle. Il reste environ 12 octets par échantillon : la sortie de l'émetteur, la copie faite par `signalAvantBruit()` et la sortie du canal. Le test 21 de `runTests` vérifie ce gain de manière automatique.

Les courbes de ce rapport avaient été calculées en tranches (200 000 bits sans codeur, 60 000 avec) pour tenir en mémoire ; avec le nouveau stockage, des tranches trois fois plus grandes tiennent dans la même mémoire.

30.3 Autres pistes

- **Éviter la copie du signal dans le canal.** `signalAvantBruit()` recopie le signal reçu avant d'ajouter le bruit ; sans écho, le canal pourrait lire directement le tableau de l'émetteur, ce qui économiserait encore 4 octets par échantillon.
- **Bruit gaussien.** La méthode de Box–Muller donnée en cours fournit deux échantillons indépendants (cosinus et sinus) pour deux tirages uniformes ; nous n'utilisons que le cosinus. Utiliser les deux diviserait par deux les appels au générateur et aux fonctions logarithme et racine carrée.
- **Semences.** Dériver deux semences (`seed` et `seed + 1`) pour la source et le canal, limite déjà relevée aux étapes 3 et 4.
- **Sondes.** L'affichage (`-s`) d'un signal de plusieurs millions d'échantillons est lent et peu lisible ; il est à réserver aux messages courts.

31 Conclusion générale

Étape 3. Le canal à bruit blanc additif gaussien est validé : bruit gaussien, centré, d'écart-type conforme à E_b/N_0 . Le récepteur, qui décidait sur un seul échantillon, perdait 14,8 dB par rapport à l'optimum.

Étape 4. Le récepteur à filtre adapté supprime cette perte : le TEB mesuré atteint la borne théorique pour les trois formes d'onde (gain de 14,8 dB en NRZ, 10 dB en RZ). Le canal à trajets multiples est intégré avec l'option `-ti` ; ses effets, avec et sans bruit, sont prédits par une théorie exacte de la chaîne.

Étape 5. Le codage de canal ($0 \rightarrow 010$, $1 \rightarrow 101$) est intégré avec l'option `-codeur`. Il divise le TEB par 9 à 2 dB et par 27 à 4 dB, soit un gain de 3,4 dB à rapport signal à bruit par bit émis constant, pour les trois formes d'onde. Ce gain se paie par un débit utile divisé par trois : à énergie égale par bit d'information, le code perd 1,4 dB. Il reste efficace avec un écho en présence de bruit, mais ne corrige pas les erreurs systématiques dues à un écho fort. Pour la question de la synchronisation, l'option `-decalage` et une fenêtre interactive permettent de désynchroniser le récepteur : sans bruit, le NRZ tolère un décalage de 14 échantillons sur 30, le RZ seulement 4 ; au-delà, le code rend l'erreur catastrophique (message inversé), mais il permet de la détecter, puisque la moitié des paquets reçus ne sont plus des mots de code. Montrer le gain sur la récupération du rythme demanderait de modéliser une dérive d'horloge et un récepteur qui se recalcule sur les transitions. Enfin, le signal analogique est désormais rangé dans des tableaux de `float` : une simulation de 200 000 bits avec codeur passe de 1 074 à 294 Mo de mémoire et deux fois moins de temps, pour des résultats identiques.

Aux étapes 4 et 5, les 958 points de TEB simulés concordent tous avec la théorie à moins de 2,5 écarts-types statistiques, et des tests automatiques vérifient désormais la *valeur* du TEB, et plus seulement le bon déroulement d'une simulation. Les suites naturelles sont un égaliseur contre les trajets multiples, et un décodage souple ou un code de meilleur rendement pour obtenir un véritable gain de codage.

Annexe – Reproduire les figures

```
./compile
python3 rapport/etape4/courbesTEB.py # figures de l'étape 4
python3 rapport/etape5/courbesCodage.py # figures de l'étape 5
python3 rapport/etape5/courbesCodage.py --recalculer # relance les
simulations (quelques minutes)
```

```
python3 rapport/etape5/courbesDecalage.py # figures de la désynchronisation
```

Sans option, les scripts réutilisent les mesures enregistrées au format CSV dans `rapport/etape4/donnees/` et `rapport/etape5/donnees/` (commande, nombre de bits, TEB simulé et théorique de chaque point). Les figures de l'étape 3 sont celles du rapport d'origine (`rapport/etape3/figures/`).

Références

- [1] J. G. Proakis, M. Salehi, *Digital Communications*, 5e éd., McGraw-Hill, 2008 – chap. 4 (détection optimale, filtre adapté), chap. 7 (codes en blocs linéaires) et chap. 9 (transmission sur canaux à bande limitée, interférence entre symboles, égalisation).
- [2] B. Sklar, *Digital Communications: Fundamentals and Applications*, 2e éd., Prentice Hall, 2001 – chap. 3 (détection en bande de base, signaux antipodaux et unipolaires) et chap. 6 (codage de canal, gain de codage).
- [3] IMT Atlantique, SIT213 – énoncé de l'atelier logiciel, commande unique du simulateur, fiche de la séance 4 « Canal à trajets multiples et bruité », TP5 « Codage de canal ».